

CAMP-VS 虚拟染色竞赛项目技术报告

从 DAPI 图像到 CD68 虚拟染色的完整工程解析

基于当前项目源码、配置与本地官方数据审计结果整理

源码核对日期：2026 年 8 月 23 日

摘要

本文面向第一次接触医学图像、深度学习和命令行的读者，完整说明 CAMP-VS 虚拟染色竞赛工程。项目把一张 DAPI 荧光显微镜图像作为输入，预测同一位置的 CD68 标记图像。本文从生物学背景讲起，逐步解释数据目录、ROI 分组、manifest、PyTorch DataLoader、Sobel 结构提示、四尺度 NAF 编解码器、余弦原型混合、任务适配器、固定权重复合损失、混合精度训练、EMA、三域验证、可选 D4 测试时增强和提交 ZIP 的产生过程。

报告不是根据模型名称猜测而写，而是逐项核对了当前工作区中的 README.md、docs/AUTODL_RUN.md、默认/初赛/resolved-local 三份 YAML 配置、CLI、训练器、验证器、模型、损失、数据集、指标及提交代码，并读取了本地官方数据审计结果。因此，本文会特别指出三项容易被误解的事实：第一，本地 JPG 在磁盘上是 RGB，只是三个通道逐像素完全相同；第二，configs/resolved_local.yaml 已把这种审计结论解析为 1 个逻辑输入/输出通道，但提交时仍恢复为原始 RGB 存储语义；第三，本地 JPG round-trip 指标是接近最终文件形态的代理指标，但没有官方评测器源码时，不能宣称它与平台算法完全相同。

阅读方法。如果你只学过计算机专业的机器学习和深度学习课程，请不要跳过第 1 节。这一节先建立“组织-ROI-patch-细胞-像素”的共同语言，再解释五个 marker、荧光强度、监督标签和评价层级；理解这些概念后，后面的网络结构和损失函数才不只是公式。如果你只想在 AutoDL 上跑通项目，可直接阅读附录“从零到提交的操作清单”。文中“当前配置”专指 configs/default.yaml 与 configs/initial_round_cd68.yaml 合并后的实际设置。

目录

1	引言与项目背景	6
1.1	什么是染色，为什么显微镜图像需要它	6
1.2	H&E、IHC、免疫荧光不是同一种数据	6
1.3	虚拟染色是什么	7
1.4	DAPI 与 CD68 分别是什么	7

1.5	荧光显微镜图像与 patch	7
1.6	竞赛目标与图像到图像翻译	8
1.7	先建立空间层级：组织、ROI、patch、细胞和像素	8
1.8	一个细胞里有哪些结构，marker 为什么出现在不同位置	9
1.9	本项目五个通道的医学含义	9
1.10	把医学概念放回一张真实的五通道 patch	10
1.11	本项目的 label 到底是什么	12
1.12	配对与配准：同名文件为什么重要	13
1.13	从光子到 Tensor：计算机实际接收了什么	14
1.14	模型学的是条件预测，不是把不可见信息恢复出来	14
1.15	像素级、细胞级、patch 级与 ROI 级评价	15
1.16	SSIM、PSNR 与医学意义应该怎样对应	15
1.17	读后续网络章节前应掌握的最小知识清单	16
2	命令行入门：为什么用 CLI 而不是 python train.py	16
2.1	CLI 是什么	16
2.2	把命令逐段拆开	17
2.3	为什么不是直接运行 train.py	18
2.4	从入口到训练器：autodl-run 内部流程	18
2.5	autodl-submit 内部流程	19
3	数据集格式与加载	19
3.1	本地官方数据的真实目录与数量	19
3.2	真实样本显示了哪些数据分布特征	20
3.3	“RGB 文件”和“灰度内容”不是同一概念	21
3.4	文件名、ROI 与二维坐标	21
3.5	manifest 是什么	22
3.6	按 ROI 划分训练集与验证集	22
3.7	Dataset 与 DataLoader 如何把 JPG 送入模型	22
3.8	PairedTransform 为什么必须“成对”	23
4	模型架构详解	23
4.1	先建立整体直觉：这是一个“先压缩理解，再放大还原”的网络	23
4.2	输入预处理与 Sobel 结构提示	24
4.3	NAF 局部编码器	24
4.4	瓶颈处的共享/任务余弦原型混合	25
4.5	解码器与 skip connection	26
4.6	marker conditioning、FiLM 与 adapter	26
4.7	直接输出头：当前没有 base/detail 与强度校准	27

目录	3
4.8 Deep Supervision	27
4.9 当前实际模型规模	27
5 数据流向详解	28
5.1 用一个真实 patch 对照源码路径	28
5.2 训练数据流	29
5.3 验证数据流	30
5.4 测试与推理数据流	31
6 训练详解	31
6.1 epoch 与 batch: 把“看很多遍数据”说清楚	31
6.2 一次 batch 内发生什么	32
6.3 梯度累积与 effective batch	32
6.4 AMP 混合精度	32
6.5 AdamW、学习率、warmup 与梯度裁剪	33
6.6 EMA: 不是另一个独立模型	33
6.7 当前固定权重复合损失	33
6.8 channels-last、cuDNN benchmark 与 float32 matmul precision	34
6.9 进度条、异步指标、profiler 与 torch.compile	34
6.10 OOM 恢复	35
6.11 验证频率、checkpoint 与早停	35
7 验证详解	35
7.1 训练和验证的根本差异	35
7.2 SSIM 与 PSNR	36
7.3 float、uint8 与 JPG 三域	36
7.4 逐图、ROI、最差样本与分层聚合	36
7.5 bootstrap 的真实状态	37
7.6 raw、EMA 与 D4 的选择	37
8 测试与提交详解	37
8.1 为什么 test 不能显示本地分数	37
8.2 D4、权重来源与 JPEG 保存	37
8.3 提交目录与命名	38
8.4 validate-submission 检查什么	38
9 配置系统: 怎样读懂并安全修改 YAML	39
9.1 YAML 是什么	39
9.2 配置合并优先级	39
9.3 当前初赛配置逐项说明	40

9.4 GPU 加速选项与边界	41
9.5 当前通道数的一个重要事实	41
9.6 新手修改配置的安全原则	41
10 项目文件结构与模块职责	42
10.1 从根目录看项目	42
10.2 包入口与公共模块	42
10.3 数据模块	43
10.4 模型模块	43
10.5 损失与指标模块	45
10.6 训练、推理与实验引擎	45
10.7 提交与工具模块	46
10.8 outputs、artifacts 与 log 的区别	47
11 总结：从一张 DAPI 到竞赛 ZIP	47
A AutoDL 从上传到训练的保姆级流程	48
A.1 第 0 步：确认上传目录	48
A.2 第 1 步：激活环境并让 Python 找到项目	48
A.3 第 2 步：可选的 3 epoch 流程检查	49
A.4 第 3 步：120 epoch 正式训练与验证	49
A.5 第 4 步：读取验证分数	50
A.6 第 5 步：对 test 推理并生成提交包	50
A.7 第 6 步：下载哪些文件	51
B 常见问题与排错顺序	51
C 实现事实核对清单	52

插图

1 医学显微图像的五個层级。左侧更接近独立生物/实验单位，右侧更接近神经网络直接处理的数值。	8
2 五类通道的典型亚细胞位置示意。marker 是分子探针，不是严格一一对应的细胞类别。	9
3 官方训练集同一位置的五个真实 marker 通道及 DAPI-CD68 教学伪彩叠加。	11
4 同一真实 patch 的全图与局部放大。DAPI 主要显示核，CD68 信号更多位于核周围和胞质区域。	12
5 autodl-run 的命令解析与主要调用链	18

6	按 CD68 全图平均强度选取的低、中、高活动度真实配对样本。这里的“活动度”是计算机统计量，不是临床阳性分级。	20
7	当前初赛配置的 MultiMarkerRestorer 主干。图中按实际单逻辑通道标注输入/输出；prototype 与 task adapter 开启，context、cross-attention 和 base/detail 关闭。	24
8	NAFBlock：两次归一化、两条门控残差路径和零初始化残差缩放	25
9	MultiTaskPrototypeMixer：共享/任务原型注意力与残差融合	26
10	一个真实配对 patch 从 JPEG、Dataset、MultiMarkerRestorer 到损失与提交文件的代码路径。	28
11	训练时从 JPG 到模型参数更新的数据流	30
12	验证时的三域指标与 ROI 聚合数据流	31

表格

1	常见组织成像方式与本项目数据的区别	6
2	本项目五个通道的入门解释	10
3	数据样本中的字段与“标签”含义	13
4	不同评价层级的含义	15
6	本地官方数据的存储与逻辑通道事实	21
7	当前 manifest 划分结果	22
8	局部编码器各尺度	25
9	当前单逻辑通道 MultiMarkerRestorer 的顶层参数量	27
10	真实 patch 在代码中的表示变化	29
11	当前固定损失的重建项权重	34
12	initial_round_cd68.yaml 的关键生效配置	40
13	src/virtual_staining 顶层模块	42
14	src/virtual_staining/data 模块	43
15	src/virtual_staining/models 模块	44
16	损失和指标模块	45
17	src/virtual_staining/engine 模块	45
18	提交和工具模块	46
19	建议从 AutoDL 下载的产物	51
20	新手常见故障	52
21	截至本报告生成时的实现事实	52

1 引言与项目背景

1.1 什么是染色，为什么显微镜图像需要它

人体组织中的细胞通常接近透明。直接把一块薄薄的组织放到显微镜下，很多结构并不会自动呈现出清晰的颜色。实验人员会使用能够与特定分子结合的染料或抗体，把原本不明显的结构“点亮”。这件事可以类比为给城市地图加图例：道路、河流、医院本来都在地图上，但用不同颜色标出后，人才能快速分辨它们。

免疫组织化学染色（Immunohistochemistry, IHC）利用抗体识别组织中的目标蛋白。传统 IHC 往往需要制备切片、加入抗体试剂、孵育、清洗和显色，过程可能持续数小时，试剂和人工都有成本，而且一次实验会对样本产生不可逆处理。若希望观察多个标记物，通常还要进行更多实验。

1.2 H&E、IHC、免疫荧光不是同一种数据

初学者容易把所有“有颜色的病理图”都叫染色图，但它们的成像机制和计算含义不同。表 1 给出最小区分。本项目文件夹按 DAPI 与四个蛋白 marker 分通道保存，像素近似灰度强度；在缺少更完整的官方实验协议时，报告把它们作为**配准的荧光强度通道**处理，不自行猜测具体显微镜型号、荧光团或抗体克隆。

表 1: 常见组织成像方式与本项目数据的区别			
方式	实验上显示什么	计算机看到什么	与本项目的关系
H&E	苏木精偏向细胞核、伊红显示胞质和基质，是常规组织形态染色	通常是彩色 RGB 图，颜色由两种染料混合	本项目没有 H&E 输入，不能假设网络看到了其丰富组织颜色。
IHC	抗体识别目标蛋白，再用显色反应显示阳性区域	常见为明场彩色图，需处理染料颜色分离	CD68 等也常用于 IHC，但本项目磁盘图是独立 marker 强度通道，不应套用普通彩色 IHC 的颜色规则。
免疫荧光	抗体携带/连接荧光信号，不同分子由不同通道记录	每个通道本质是空间强度矩阵，可单独灰度显示，也可伪彩叠加	本项目的数据组织最接近这种“DAPI 加多个 marker 通道”的计算形式。
虚拟染色	不新增化学探针，由模型预测目标通道	输入张量经过神经网络得到连续强度图	预测图只是对实验图的统计估计，不能反过来当作真实做过一次染色。

1.3 虚拟染色是什么

虚拟染色 (Virtual Staining) 并不是让计算机真的把染料滴到玻片上，而是训练一个人工智能模型：给它一种已经拍摄好的图像，让它预测同一位置在另一种标记下应该呈现的像素强度。对本项目来说，可以把模型想象成一个非常复杂的“逐像素翻译器”：输入一张 DAPI 图，输出一张 CD68 图，输入和输出的尺寸、位置一一对应。

它的潜在价值包括：

- 减少重复染色所需的时间和试剂；
- 在不再次处理样本的前提下，探索另一种标记物的可能分布；
- 同一张 DAPI 图理论上可作为多个目标 marker 的共同输入；
- 预测是软件过程，可以批量运行并保留可复现的参数与日志。

需要强调的是，竞赛模型输出是**算法预测**，不是经过临床验证的真实检测结果。它可以用于竞赛、研究和方法开发，但不能因为图像“看起来像”就直接替代病理诊断。

1.4 DAPI 与 CD68 分别是什么

DAPI (4',6-diamidino-2-phenylindole, 4',6-二脒基-2-苯基吲哚) 是一种常用的蓝色荧光 DNA 染料。它与双链 DNA 的富含 A-T 区域结合后荧光明显增强，所以在荧光显微镜下常被用作细胞核复染。简单地说，DAPI 图像主要告诉我们“细胞核在哪里、密度如何、形状怎样”。DAPI 的常见性质可参考 Thermo Fisher 的技术说明 [8]。

CD68 (Cluster of Differentiation 68) 是一种主要位于内体/溶酶体系统的糖蛋白，在单核细胞和组织巨噬细胞中常见。病理研究中常用 CD68 抗体辅助观察巨噬细胞相关信号和炎症细胞浸润；Human Protein Atlas 也给出了相应的组织表达资料 [9]。但是 CD68 并非“只在巨噬细胞出现”的绝对身份证，一些非髓系细胞也可能表达，因此单凭 CD68 阳性不能完成细胞身份判定 [10]。为了入门，可以暂时把 CD68 图理解成“巨噬细胞相关信号的空间热力图”，同时牢记这只是近似说法。

只看 DAPI 并不能直接读取 CD68。模型需要从大量同位置的 DAPI-CD68 配对样本中学习统计规律：哪些核形态、密度和周围纹理经常与 CD68 信号共同出现。这个映射不是确定的化学公式，因此虚拟染色本质上是一个有不确定性的预测问题。

1.5 荧光显微镜图像与 patch

荧光显微镜图像 (Fluorescence Microscopy Image) 记录荧光分子在不同位置发出的光强。存储到文件后，每个像素是一个数字：数值较大通常代表该位置的信号较强。项目内部会把 8 位像素 0 ~ 255 转成浮点数 0 ~ 1，以便神经网络计算。

一张完整组织切片可能非常大，不能总是整张送入 GPU。因此数据制作者会把大图裁成许多固定大小的 **patch (图块)**。本地官方数据的每个 patch 是 256 × 256 像素。文件名如 ROI000_03_12.jpg，可拆成：

- ROI000: 第 0 个感兴趣区域;
- 03: 该 patch 在大区域网格中的行坐标;
- 12: 列坐标;
- .jpg: JPEG 文件格式。

ROI (Region of Interest, 感兴趣区域) 可以理解为从大切片中选出一块连续区域。同一 ROI 里的相邻 patch 可能原本就在大图上彼此相邻, 因此它们并不是完全独立的样本。

1.6 竞赛目标与图像到图像翻译

项目的核心任务是: 对每张测试 DAPI patch 生成同尺寸的 CD68 patch。模型学习的是

$$f_{\theta} : \mathbf{x}_{\text{DAPI}} \longrightarrow \hat{\mathbf{y}}_{\text{CD68}}, \quad (1)$$

其中 θ 是模型参数, $\mathbf{x}$ 是输入图, $\hat{\mathbf{y}}$ 是预测图。因为输入和输出都是图像, 而且像素位置需要对应, 所以它属于图像到图像翻译 (Image-to-Image Translation), 更具体地说也属于图像恢复/回归问题。

本地验证主要观察 SSIM (Structural Similarity, 结构相似性) 与 PSNR (Peak Signal-to-Noise Ratio, 峰值信噪比)。SSIM 更关注亮度、对比度和局部结构是否相似, PSNR 则由均方误差推导。两者通常都是越高越好, 但它们不能替代医学专家判断。

1.7 先建立空间层级: 组织、ROI、patch、细胞和像素

计算机视觉教材常从一张独立的图像讲起; 医学显微图像却有天然的层级结构。一张 JPG 并不等于一个独立患者, 也不一定等于一个独立生物样本。图 1 给出了本项目最重要的空间层级。

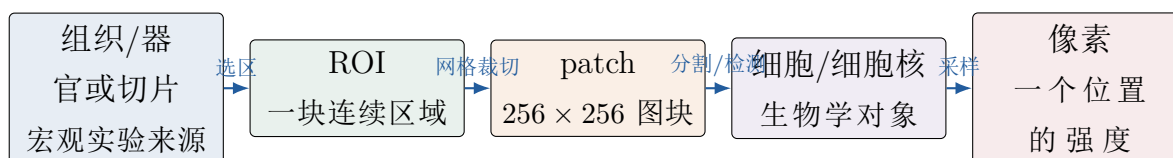


图 1: 医学显微图像的五個层级。左侧更接近独立生物/实验单位, 右侧更接近神经网络直接处理的数值。

- 组织/切片层: 实验者从某个生物样本得到组织切片。一张切片可包含大量细胞和多种组织结构。当前公开目录没有患者或玻片标识, manifest 的 organ 也为 unknown, 所以本报告不能把“一个 ROI”冒充“一个患者”。

- **ROI 层**: ROI 是切片中一块连续的感兴趣区域。不同 ROI 通常比同一 ROI 内的两个 patch 更接近统计独立，因此训练/验证划分必须优先按 ROI 分组。
- **patch 层**: 为了放入 GPU，大区域被切成 256×256 小图。文件名里的行、列坐标描述它在 ROI 网格中的位置；相邻 patch 可能共享相似的组织纹理和细胞群。
- **细胞层**: 病理学家关心的对象往往是细胞，例如“有多少 CD68 相关阳性细胞”“信号是否围绕某些细胞核出现”。要从图像得到细胞级量，通常还要先分割细胞核或细胞区域。
- **像素层**: 神经网络最终输出每个位置的亮度。像素是采样值，不是一个细胞，也不是一个分子。一枚细胞核通常覆盖许多像素，一个像素也可能混合多个亚细胞结构和背景光。

这解释了“像素级指标很好但细胞级结论可能错误”：模型可能把一个真实的亮点轻微平移或摊成一片较弱的光。大部分像素的误差仍然不大，但被判为阳性的细胞数、阳性细胞位置或局部表达强度已经发生变化。

1.8 一个细胞里有哪些结构，marker 为什么出现在不同位置

理解 marker 前，先把细胞极度简化为四个区域：**细胞核**储存 DNA；**细胞质**包含细胞器和骨架；**细胞膜**把细胞与外界分隔并承载许多受体；**细胞外基质**位于细胞之间并支撑组织。真实细胞没有规则圆形边界，而且二维切片只截取三维细胞的一层。图 2 只是空间位置的教学示意，不代表数据中的真实颜色、比例或细胞类型。

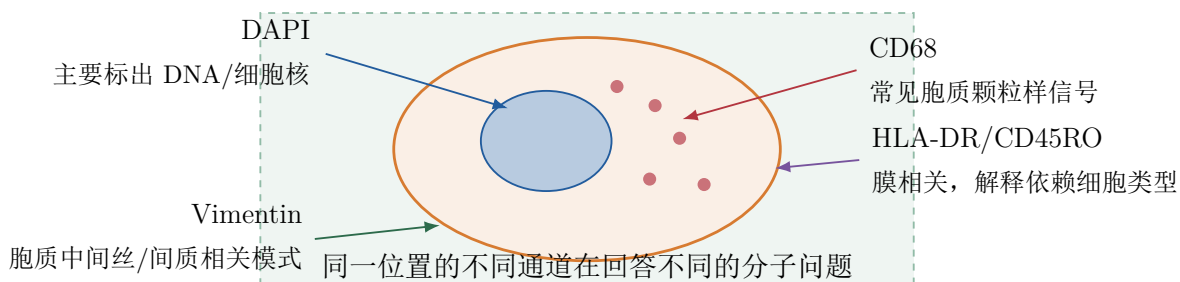


图 2: 五类通道的典型亚细胞位置示意。marker 是分子探针，不是严格一一对应的细胞类别。

1.9 本项目五个通道的医学含义

训练目录中同一个 basename 在五个 marker 文件夹各有一张图。它们可看作同一空间位置的五种“观察方式”。表 2 中的“常见关联”用于建立直觉，不应被当成临床诊断规则。

表 2: 本项目五个通道的入门解释

通道	主要分子/结构	常见医学关联	图像中可期待的空间形态	不能直接推出什么
DAPI	与双链 DNA 结合的核染料	提供细胞核位置、大小、形状和局部核密度	一个个核样亮区域；不同细胞的核形态可不同	看不见完整细胞膜；不能仅凭核图确定某蛋白一定表达
CD68	内体/溶酶体相关糖蛋白	常用于辅助观察单核细胞/巨噬细胞相关信号	常见胞质内、核周或颗粒样信号，而不是填满细胞核	不是巨噬细胞的绝对专属身份证；阳性不自动等于疾病诊断
CD45RO	CD45/PTPRC 的一种异构体	常与人类记忆/活化 T 细胞表型相关，但不同免疫亚群需联合其他 marker 判断 [11]	作为白细胞表面蛋白，常呈细胞轮廓/膜相关分布	不能只用该通道确定所有 T 细胞亚型、抗原特异性或功能状态
HLA-DR	人类 MHC II 类分子	抗原呈递细胞表面常见，也可在炎症刺激下于其他细胞上调 [12]	以细胞膜相关信号为主，也受胞内运输影响	不是单一细胞类型专属；强信号不能直接等同某种疾病
Vimentin	III 型中间丝细胞骨架蛋白	常见于间充质/基质相关细胞，且在多种细胞环境中表达	细胞质内纤维状、网状或弥散结构；不与细胞核完全重合 [13]	不能把所有 Vimentin 阳性区域都判成同一种细胞或固定生物过程

因此，从 DAPI 预测 CD68 等通道并不是简单地给细胞核“换颜色”。例如，一个 DAPI 核周围是否出现 CD68 颗粒，与细胞身份和状态有关；而 DAPI 本身没有直接测量该蛋白。网络只能利用训练集中反复出现的核形态、密度、组织纹理和蛋白信号之间的统计相关性。

1.10 把医学概念放回一张真实的五通道 patch

前面的示意图只说明“marker 可能出现在哪里”。图 3来自本地官方训练集的同个真实空间位置 R0I007_09_11，能更直观地说明五个通道并不是五张无关照片。前五格直接显示原始 uint8 灰度，没有对每个通道单独拉伸对比度；右下角只是教学用伪彩叠加，把 DAPI 映射为蓝色、CD68 映射为红色。数据文件本身并没有这些蓝红颜色。

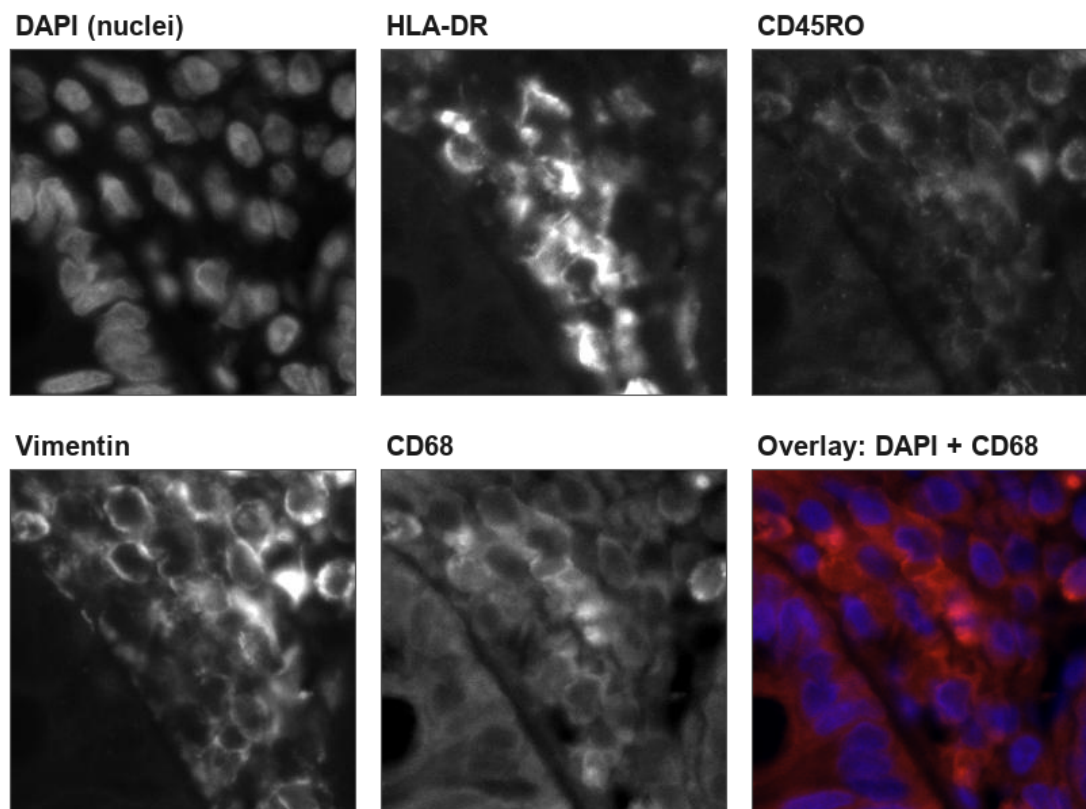


图 3: 官方训练集同一位置的五个真实 marker 通道及 DAPI-CD68 教学伪彩叠加。

读图时可以按下面的顺序观察：

1. **先看 DAPI：**一个个椭圆或不规则的亮区主要对应细胞核。它能提供核的位置和密度，但没有画出完整细胞边界。
2. **再横向比较 marker：**HLA-DR 在图中部有较集中的亮结构，CD45RO 的强度和纹理又不同；Vimentin 与 CD68 在不少位置呈胞质/轮廓相关信号。它们共享空间坐标，却不是 DAPI 的复制品。
3. **最后看叠加图：**蓝色核与红色 CD68 信号经常相邻、包围或部分重叠，而不是严格像素重合。这正是“核层级信息”和“胞质/蛋白层级信息”的区别。

这些观察只能描述本 patch 的图像形态，不能据此给每个细胞下临床诊断。例如，不能把所有红色区域直接数成巨噬细胞，也不能因为某个核周围没有明显红色就断言该细胞绝对不表达 CD68。严谨的细胞级结论还需细胞分割、多 marker 联合判定、实验阈值和病理专家复核。

图 4 进一步放大同一 patch 中 CD68 平均强度较高的 96×96 窗口。红框和裁剪位置由脚本根据 CD68 局部均值自动选择，并不是人工挑出的“典型阳性细胞”或医学标注。

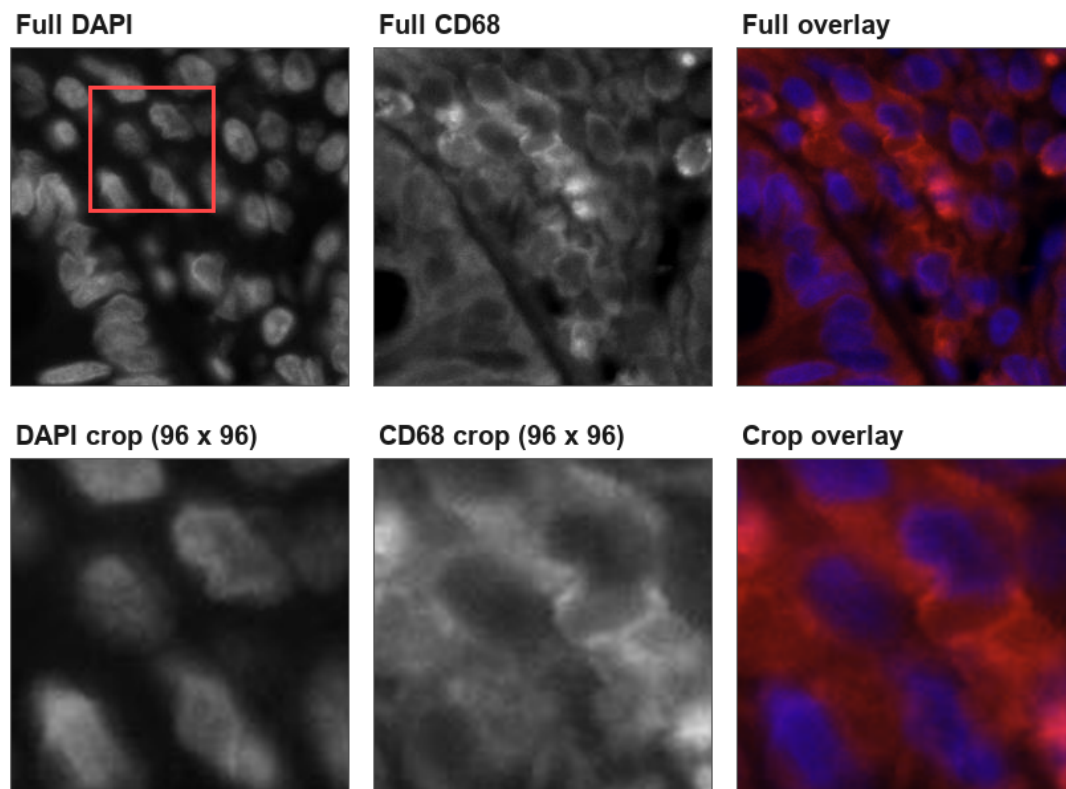


图 4: 同一真实 patch 的全图与局部放大。DAPI 主要显示核，CD68 信号更多位于核周围和胞质区域。

局部图也解释了为什么模型难：DAPI 给出蓝色核的形状，却没有直接告诉网络红色信号应当沿哪个核的哪一侧增强。网络只能从训练样本中学习常见条件关系；当 DAPI 外观相似而 CD68 状态不同时，像素回归会倾向输出较平滑的平均结果。

1.11 本项目的 label 到底是什么

计算机课程里，label 常指“猫/狗”或 0/1。本任务不同：主要监督信号是一整张目标 marker 图，称为 **稠密标签 (dense target)**。每个目标像素都是一个回归值，而不是给整个 patch 一个类别。

表 3: 数据样本中的字段与“标签”含义

信息	计算机表示	含义与用途
DAPI 输入	$H \times W \times C$ 图像	模型可见的条件信息；当前 JPG 存储为 RGB，但三通道相同。
目标图	同尺寸连续强度图	train 中 HLA-DR、CD45RO、Vimentin、CD68 都存在；当前初赛配置只把 CD68 作为训练目标。
marker 名	字符串/任务 ID	告诉多任务模型要预测哪种蛋白通道；它不是患者诊断。
ROI、row、col	分组与整数坐标	用于防止泄漏、恢复邻域和按区域汇总；不直接参与当前默认 context-disabled 模型。
split	train/val/test	train 用于更新权重，val 用于选模型，test 只有 DAPI 且隐藏目标由竞赛平台保管。
organ	元数据字符串	当前本地 manifest 为 unknown ；在没有官方元数据时不能从文件夹结构臆造器官标签。

本地 official 数据的结构是：训练集中每个 marker 各 6296 张、可组成 6296 组同名配对；按真实 ROI 分组后得到 5004 个 train patch 和 1292 个 val patch。测试目录有 1346 张 DAPI，但没有任何目标图。所以 val 分数是本地可计算的开发指标，test 分数必须把预测提交到平台后才能得到。

还要区分“图像强度”和“蛋白分子数”。荧光图中的亮度受染色效率、曝光、探测器、背景、自发荧光、漂白和饱和等因素影响；若没有逐批校准，像素值通常只是相对强度，不等于一个可以跨实验直接比较的绝对分子计数 [14]。因此网络拟合的是数据制作流程下的目标图像分布，而不是把 0 ~ 255 机械地翻译成“多少个蛋白分子”。

1.12 配对与配准：同名文件为什么重要

监督学习需要知道哪一张 DAPI 对应哪一张目标图。本工程用 marker 文件夹和规范化后的同名 stem 建立配对，例如：

```
train/DAPI/ROI000_03_12.jpg
train/CD68/ROI000_03_12.jpg
```

相同的 ROI000_03_12 表示二者应来自同一网格位置；这叫**空间配对**（spatial pairing）。但“文件名相同”不等于所有边缘都精确重合：制样、通道采集、光学系统和压缩仍可能造成轻微位移或噪声。本地审计得到目标通道与 DAPI 的中位平移为 0 像素，说明没有发现系统性整体错位，所以默认不做配准；这不代表每个细胞的所有亚细胞信号都应该与 DAPI 核重合。CD68 本来就主要不是核内信号。

若训练/验证把同一 ROI 的相邻 patch 随机拆开，模型可能在验证时看见几乎相同的组织环境，导致分数虚高。这叫**数据泄漏 (data leakage)**。本项目按 ROI 分组，当前 train 与 val 的 ROI 不重叠。对于未来含患者或玻片 ID 的数据，应该优先按患者/玻片分组，因为它们比 ROI 更高一层。

1.13 从光子到 Tensor：计算机实际接收了什么

荧光显微镜把某个波段的光转换成探测器读数，保存前又可能经过曝光、增益、位深转换和压缩。进入当前工程后，一张图经历下面的表示变化：

1. 磁盘上的 JPEG 解码为 $256 \times 256 \times 3$ 的 uint8 数组，元素为整数 $0 \sim 255$ ；
2. 图像库按 RGB 顺序读取。审计发现三通道逐像素相同，所以医学信息上是一个灰度强度通道；
3. 数据集把像素除以 255，变成 float32 的 $0 \sim 1$ ；
4. 维度从 HWC 调整为 PyTorch 的 CHW，再加 batch 维成为 $B \times C \times H \times W$ ；
5. 网络输出同范围的浮点预测；提交时先四舍五入量化回 uint8，再以固定 JPEG 参数编码。

JPEG 是有损格式。同一个浮点预测在“内存 float”“量化 uint8”“写成 JPG 后重新读取”三个域里并不完全一致，这也是项目同时报告三域指标的原因。背景、自发荧光和探测器噪声会减少有效动态范围，饱和像素则丢失超过上限的强度信息；这些都是定量荧光显微分析的常见限制 [14]。

1.14 模型学的是条件预测，不是把不可见信息恢复出来

更准确的数学说法是，训练数据从某个联合分布产生 $(X_{\text{DAPI}}, Y_{\text{marker}})$ ，模型试图学习

$$\hat{Y} = f_{\theta}(X) \approx \mathbb{E}[Y | X]. \quad (2)$$

如果两个细胞的 DAPI 外观几乎相同，但一个表达 CD68、另一个不表达，那么仅凭 DAPI 无法唯一判别。在均方误差等损失下，模型容易给出两种可能性的平均，即一个偏弱、偏平滑的信号。这不是简单增加网络参数就一定能解决的问题，而是**条件不确定性 (conditional uncertainty)**：输入中可能缺少决定目标的分子信息。

因此应避免三种过度表述：其一，“模型检测到了 CD68 蛋白”；准确说法是“模型预测了 CD68 图像”；其二，“亮点就是巨噬细胞”；准确说法是“亮点与 CD68 相关，细胞身份仍需联合形态和其他 marker”；其三，“SSIM 提高就有临床价值”；临床价值还需要外部队列、细胞级分析和专家验证。

1.15 像素级、细胞级、patch 级与 ROI 级评价

表 4 把后文经常出现的“层级”拆开。一个指标属于哪一层，决定了它能回答什么问题。

表 4: 不同评价层级的含义

层级	计算对象	可以回答的问题	局限与常见指标
像素级	每个坐标的预测强度与真值强度	整体灰度和局部结构有多接近？	MSE、PSNR、SSIM；不理解细胞身份，且大量暗背景会稀释稀有亮点的错误。
细胞级	先用 DAPI 分割核/细胞，再对每个对象汇总 marker	阳性细胞比例、单细胞平均/最大强度、信号与核的距离是否合理？	依赖分割和阳性阈值；本项目当前标准 Validator 尚未计算。
patch 级	一张 256×256 图的指标或统计量	哪些局部图块最难？性能是否随亮度、密度变化？	同一 ROI 内 patch 高度相关，不能把它们全当独立样本。
ROI 级	同一 ROI 的全部 patch	一个连续组织区域的平均表现和空间一致性如何？	ROI 数通常远小于 patch 数；本项目用 ROI 聚合和成对 bootstrap 做可靠比较。
切片/患者级	同一玻片或患者的多个 ROI	方法能否跨患者泛化，是否改变患者级结论？	当前数据没有可验证的患者/玻片 ID，不能声称通过患者级验证。

一个简单反例能说明为什么要分层：真值里有 2 个强阳性细胞，预测把相同的总亮度均匀分给 10 个细胞。两张图的总强度可能相近，某些像素平均误差也未必极端，但“阳性细胞数量”和“信号集中在哪些细胞”已经完全不同。反过来，如果预测只比真值平移 1 个像素，细胞级阳性结论可能不变，像素级 MSE 却会明显增大。

1.16 SSIM、PSNR 与医学意义应该怎样对应

给真值图 Y 和预测图 $\hat{Y}$ ，均方误差为

$$\text{MSE} = \frac{1}{N} \sum_{i=1}^N (Y_i - \hat{Y}_i)^2, \quad (3)$$

当图像范围归一化为 $[0, 1]$ 时，PSNR 为

$$\text{PSNR} = 10 \log_{10} \left(\frac{1}{\text{MSE}} \right). \quad (4)$$

PSNR 高表示平均平方误差小，但它对整体亮度范围敏感，也不知道一个误差落在背景还是关键阳性细胞上。SSIM 在局部窗口比较亮度、对比度和结构 [4]，比单纯 MSE 更接近“纹理是否相似”，但它也没有细胞语义：两团形状相似却属于不同细胞的信号，SSIM 不会自动理解其医学差别。

本报告后文的 float/uint8/JPG 指标仍属于同一套像素/局部结构指标在三种文件表示上的计算，不是三个医学终点。若要进一步研究医学一致性，可在不改变竞赛主指标的前提下增加：单细胞 marker 强度相关、阳性细胞比例误差、细胞间强度排序的 Spearman 相关、ROI 内空间分布差异等。它们必须固定细胞分割方法和阈值，并报告失败样本，不能挑一张好看的热力图代替统计检验。

1.17 读后续网络章节前应掌握的最小知识清单

1. 一张 target 图是稠密回归标签，不是一个类别编号；
2. DAPI 主要描述细胞核，CD68/HLA-DR/CD45RO/Vimentin 描述的是不同分子和亚细胞位置；
3. marker 与细胞类型是“相关但不绝对唯一”的关系，细胞身份通常需要多 marker 与形态联合判断；
4. 一个 ROI 含很多相邻 patch，随机按 patch 划分会造成泄漏；
5. 像素、细胞、patch、ROI、患者是不同统计单位，不能混用样本量；
6. 荧光亮度是受实验流程影响的相对测量，不应直接称为蛋白分子数；
7. 网络预测条件分布中的常见结果，不能凭 DAPI 创造输入中完全不存在的确定信息；
8. PSNR/SSIM 衡量图像相似度，不自动证明细胞级或临床级正确；
9. test 没有公开目标，验证命令只能评估 val，平台分数才是隐藏 test 的结果；
10. JPEG 量化/压缩会改变像素，所以最终竞赛比较应重点看 JPG round-trip 指标和平台结果。

2 命令行入门：为什么用 CLI 而不是 python train.py

2.1 CLI 是什么

CLI (Command Line Interface, 命令行接口) 是通过输入文字命令控制程序的方式。图形界面像餐厅菜单，CLI 更像把“菜名、份量、备注”写成一行交给厨房。它看起

来不如按钮直观，但非常适合服务器：命令可以复制、记录、重跑，也能清楚说明使用了哪个配置和数据目录。

本项目推荐的训练命令是：

```
python -m virtual_staining.cli --log-root log autodl-run \
  --config configs/initial_round_cd68.yaml \
  --data-root AUTO --target CD68 --run-id cd68_v2_seed2026
```

2.2 把命令逐段拆开

命令片段	含义
python	启动当前环境中的 Python 解释器。AutoDL 上应先激活装好 PyTorch 的环境。
-m	告诉 Python：“后面给的是可导入模块名，请按模块运行”，而不是磁盘上的单个脚本路径。
virtual_staining.cli	Python 模块路径，对应源码 src/virtual_staining/cli.py。安装项目后，Python 能从 src/ 布局中找到它。
--log-root log	全局选项。每次 CLI 调用会在根目录 log/ 下创建轻量日志目录，记录环境、命令、配置、epoch 指标和错误；成功结束后还会打成日志 ZIP。该参数必须放在子命令之前。
autodl-run	子命令，表示执行“环境检查、数据准备、训练和验证”的一键编排。
--config ...yaml	指定本次使用的 YAML 配置。YAML 就像一张集中填写超参数的表。
--data-root AUTO	不硬编码目录，让发现程序在受限候选路径中评分并选择数据根；当前应发现 dataset/official。
--target CD68	通用训练/验证命令可用的目标参数。需要注意：当前 autodl-run 包装器实际从 YAML 的 data.targets 和 data.submit_targets 读取唯一目标；本配置本来就是 CD68，因此该参数起到说明意图的作用，但不要误以为它会覆盖包装器中的 YAML 目标。
--run-id ...	本次实验的唯一名字，同时用于输出目录和日志目录。已有非空输出目录不会被静默覆盖。

2.3 为什么不是直接运行 train.py

项目根目录并不存在一个承担全部工作的 `train.py`。代码采用标准 `src 布局 (src layout)`：可导入包位于 `src/virtual_staining/`。使用 `-m` 有三点好处：

1. Python 按包规则解析模块间导入，不依赖你碰巧站在哪个子目录；
2. 所有功能共享同一个入口，却可通过子命令区分 `train`、`validate`、`predict`、`autodl-submit` 等流程；
3. 参数解析、日志、异常退出码和配置合并方式保持一致，便于服务器自动化。

如果已经执行 `pip install -e .`，包会以 `editable`（可编辑）方式注册到当前 Python 环境；修改 `src/` 后不需要反复复制包。若不安装，也可以把根目录下的 `src` 加进 `PYTHONPATH`。

2.4 从入口到训练器：autodl-run 内部流程

图 5 展示了命令的路由关系。

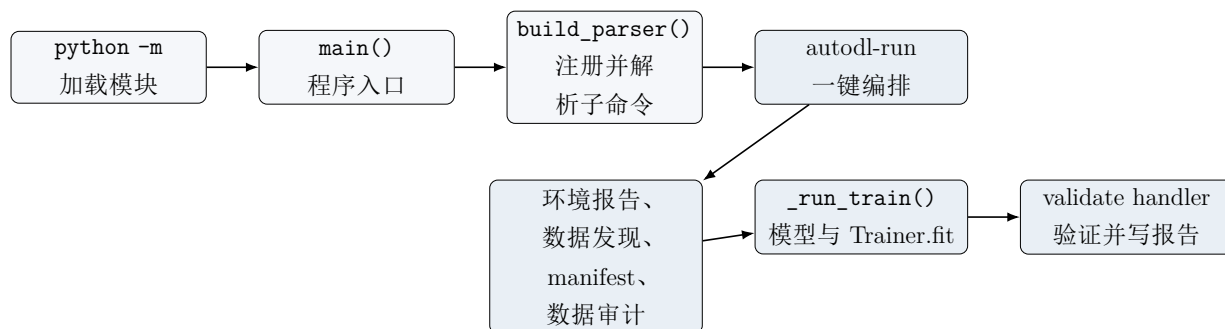


图 5: autodl-run 的命令解析与主要调用链

真实代码的执行顺序如下：

1. 模块末尾调用 `main()`；`build_parser()` 创建 `argparse` 解析器并注册所有子命令。
2. 参数解析完成后，`ExperimentLogSession.from_args()` 先在 `log/<run-id>/` 创建 `invocation`、`environment` 和 `status` 文件，并把控制台输出同时写入日志。
3. 解析器发现子命令是 `autodl-run`，调用 `command_autodl_run()`。
4. `command_env()` 收集 Python、PyTorch、CUDA、GPU、CPU 和内存信息，写入 `artifacts/environment.json`。
5. `discover_data_root()` 对有限候选目录打分；当前数据根为 `dataset/official`。
6. `_ensure_manifests()` 配对文件并建立 `train`、`val`、`test` 与 `smoke` CSV；随后 `audit_data()` 统计尺寸、通道、值域、相关性和对齐情况。

7. `_run_train()` 合并配置、构建 `DataLoader`、`MultiMarkerRestorer`、固定权重 `CompositeRestorationLoss`、`AdamW`、`Trainer` 与 `Validator`；`Trainer.fit()` 按配置训练 120 epoch。
8. 训练内部在每个验证 epoch 比较 raw 与 EMA。训练结束后，`command_validate()` 再加载最佳 checkpoint 中记录的选中权重来源，输出逐图和汇总指标；当前初赛配置的推理 TTA 为 `none`，只有显式改成 D4 才会额外执行八变换比较。
9. `pipeline_report.json` 汇总环境、数据、训练和验证路径。CLI 成功结束时，日志会复制关键轻量文件并生成 `autodl-run_log_bundle.zip`。

2.5 autodl-submit 内部流程

提交命令不会再训练。它依次做以下事情：

1. 若给了 `--checkpoint` 就使用它；否则在 `outputs/**/best_ssim.ckpt` 中选修改时间最新的文件。
2. 重新发现数据并建立 test manifest，确认 test 不是空表。
3. `command_predict()` 根据 checkpoint 内保存的模型架构和 `ImageSpec` 构建模型，选择 raw 或 EMA 权重，按需要执行 D4 TTA，并把预测保存到 `predictions/CD68/`。
4. `command_make_submission()` 将预测复制为规定命名，构建 `results/test/CD68/` 并压缩为 ZIP。已经是 JPEG 的预测会直接复制，避免二次有损编码。
5. `validate_submission()` 检查文件名、数量、尺寸、模式、像素范围、纯黑/纯白异常、ZIP 层级和 CRC。任一硬错误都会令命令返回非零退出码。

3 数据集格式与加载

3.1 本地官方数据的真实目录与数量

当前工作区的真实目录为：

```
dataset/official/
|-- train/
|   |-- DAPI/          6296 JPG
|   |-- HLA-DR/        6296 JPG
|   |-- CD45RO/        6296 JPG
|   |-- Vimentin/      6296 JPG
|   `-- CD68/          6296 JPG
`-- test/
    `-- DAPI/          1346 JPG
```

训练目录每个 marker 的文件名一一对应，总计 6296 组配对样本。当前初赛配置只选择 CD68，因此每次训练实际读取 DAPI 与 CD68；其余三个目标仍在磁盘和 manifest 中，但不参与本次单目标 loss。test 只有 1346 张 DAPI，没有 CD68 标签，所以本地无法为 test 计算 SSIM 或 PSNR。

3.2 真实样本显示了哪些数据分布特征

仅展示一张“好看的图”容易造成误解。图 6 先对 6296 张训练 CD68 图计算归一化平均强度，再选取接近第 10、50、90 百分位的三个样本。每行左侧是模型输入 DAPI，中间是训练目标 CD68，右侧是教学伪彩叠加；mean 是 CD68 全图平均强度，>0.25 表示强度超过 0.25 的像素比例。

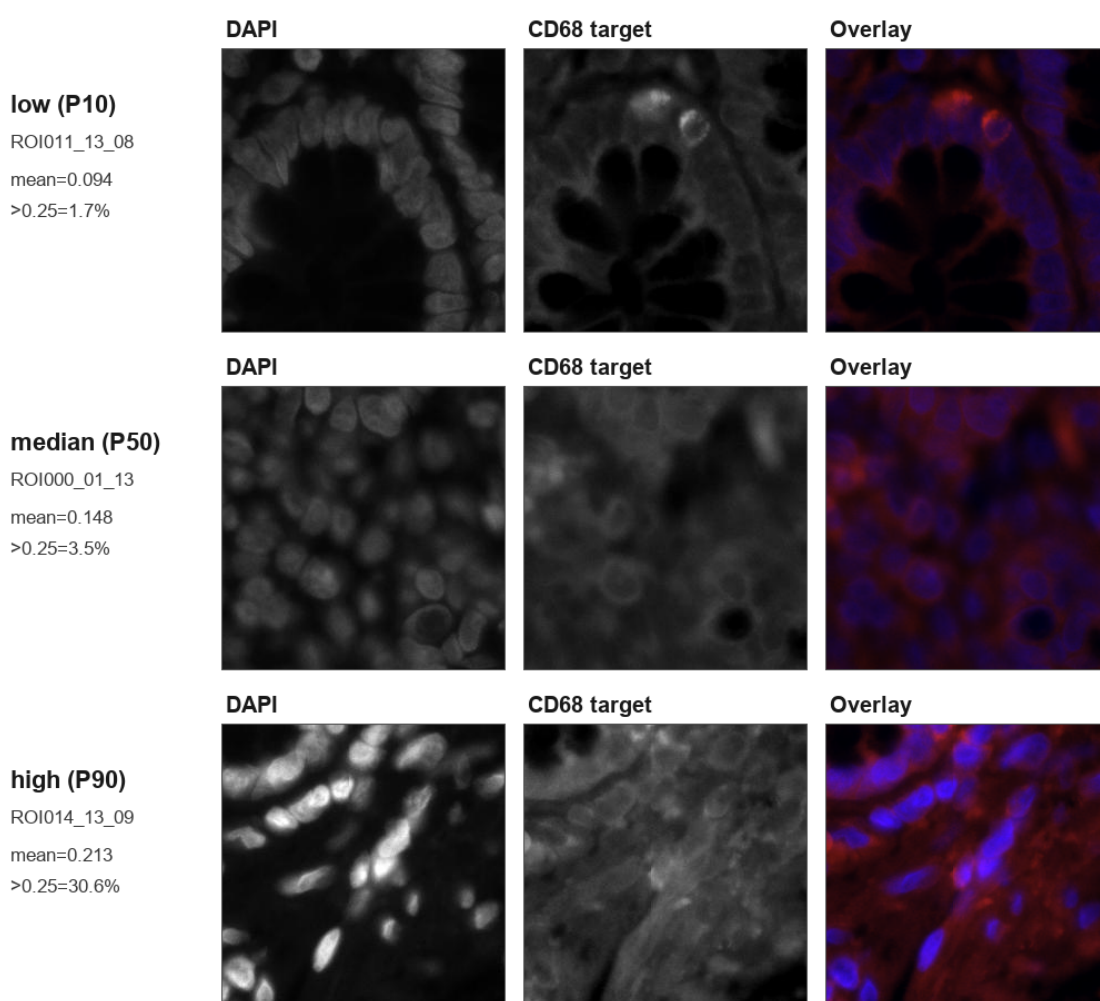


图 6: 按 CD68 全图平均强度选取的低、中、高活动度真实配对样本。这里的“活动度”是计算机统计量，不是临床阳性分级。

这三行揭示了四个与建模直接相关的特征：

- **目标强度分布不均匀：**不同 patch 的背景、亮区面积和局部峰值相差明显。若训练只

优化全图平均误差，大量暗背景可能压过少量但重要的亮结构。

- **DAPI 与 CD68 不是确定映射：**核密度、核亮度与 CD68 有统计联系，但从三行不能得到“DAPI 越亮，CD68 必然越亮”的单调规则。
- **标签是连续图而非二分类 mask：**CD68 目标包含从暗背景到强信号的连续值。报告中的 0.25 只用于展示像素占比，不是项目训练标签，也没有被当作临床阳性阈值。
- **结构与强度都要学习：**网络既要预测哪里出现信号，又要估计信号有多强；这也是复合损失同时使用 MSE/Charbonnier 与 SSIM、梯度、金字塔项的动机。

选择过程可由 `docs/essay/build_report_figures.py` 完整复现，具体样本、统计量和显示规则记录在 `docs/essay/figures/report_figure_provenance.json`。脚本只读取原始数据，不会覆盖、重编码或移动 `dataset/official` 中的任何文件。

3.3 “RGB 文件”和“灰度内容”不是同一概念

本地 `artifacts/data_audit.json` 的全量审计结果为：所有 DAPI 和四个目标图都是 256×256 、8 位 JPEG、PIL mode RGB、3 个存储通道；同时，三个通道之间的平均差和最大差均为 0。因此，图像内容在逻辑上是灰度，文件容器却是 RGB。

表 6: 本地官方数据的存储与逻辑通道事实

Marker	图数	尺寸	存储通道	逻辑通道	PIL mode
DAPI	6296	256×256	3	1	RGB
HLA-DR	6296	256×256	3	1	RGB
CD45RO	6296	256×256	3	1	RGB
Vimentin	6296	256×256	3	1	RGB
CD68	6296	256×256	3	1	RGB

`initial_round_cd68.yaml` 本身没有重复写通道数，但配置加载器随后合并 `configs/resolved_local.yaml`；该文件根据审计结果设置 `data.input_channels: 1`，并把四个 target 的逻辑通道数都设为 1。因此当前 Dataset 与模型实际按 `[B, 1, 256, 256]` 运行，输出也是单通道逻辑张量。这里的“单通道训练”不等于把原始文件改成 mode L：官方 JPEG 仍保持只读 RGB，提交保存时仍按 checkpoint 中记录的 RGB storage mode 恢复三个相同通道。

3.4 文件名、ROI 与二维坐标

文件名使用 `ROI{编号}_{行}_{列}.jpg`。manifest 中旧的 `parse_roi_id()` 会提取 ROI000 用于分组；更严格的 `parse_patch_coordinate()` 可进一步解析行列，供网格审计、边界/内部分类和可选邻域模型使用。当前初赛配置的 `model.context.enabled=false`，因此训练只使用中心 patch，不会偷偷拼接相邻图块。

坐标有空洞并不一定代表数据损坏。例如 ROI 网格可能只保留组织区域，背景 patch 被排除。程序只把真实存在的文件写入 manifest，不会凭空生成缺失坐标。

3.5 manifest 是什么

manifest (清单文件) 是 CSV 表格。它不存像素，只存“到哪里找图”和“这张图属于谁”。每一行包含 organ、split、ROI、patch ID、规范化 key、DAPI/目标相对路径、宽高、通道、mode、文件大小、SHA-1 指纹、是否配对及缺失标记等。这样，训练过程可以被审计：某个预测出错时，能追溯到原始文件。

路径写成相对于数据根的形式，例如 `train/DAPI/ROI000_00_00.jpg`，运行时再与数据根拼接。代码使用 `pathlib.Path`，因此支持中文和空格路径。

3.6 按 ROI 划分训练集与验证集

官方目录没有单独的 `val` 文件夹。程序先从文件名得到 25 个训练 ROI (ROI000–ROI024)，再以种子 2026 按组抽取约 20% ROI 作为本地 `val`。当前清单结果见表 7。

表 7: 当前 manifest 划分结果

集合	patch 数	ROI 数	说明
train	5004	20	用于反向传播与参数更新
val	1292	5	ROI004、ROI005、ROI006、ROI011、ROI023；不参与参数更新
test	1346	5	只有 DAPI；用于生成提交

当前泄漏报告显示 `train` 与 `val` 的 canonical key、DAPI 哈希和 group ID 交集均为空，ROI 来源为可靠的文件名正则，因而这次 ROI 分组划分是可验证的。为什么不能简单随机抽 20% patch？因为同一 ROI 的相邻 patch 很相似。如果左边 patch 进训练集、右边 patch 进验证集，模型可能靠记忆局部区域获得虚高分数。按 ROI 分组相当于把整本练习册分开，而不是把同一道题的上下半部分分别放到训练和考试中。

3.7 Dataset 与 DataLoader 如何把 JPG 送入模型

`VirtualStainingDataset.__getitem__()` 的核心步骤是：

1. 从 manifest 取一行，拼出 DAPI 和 CD68 的绝对路径；
2. PIL 解码 JPEG。灰度存储会转 L，其他存储会转 RGB；当前文件因此得到 HWC RGB 数组；
3. 因逻辑通道数为 1，对三个相同 RGB 通道取均值并四舍五入，再转为 CHW 张量、除以 255，得到 `float32` 的 `[0,1]` 值；

4. 检查 DAPI 与目标空间尺寸完全相同；
5. 训练时调用成对增强，最后再次转 float32 并截断到 [0, 1]；
6. 返回输入、targets 字典、stem、ROI、organ、split 和路径元数据。

当前 DataLoader 使用 `batch_size=8`、`num_workers=0`。虽然 default/初赛配置分别写过 `pin_memory=true` 与 `persistent_workers=true`，但加载器只有在 CUDA 可用时才真正 pin memory，并且 workers 为 0 时会强制令 persistent 为 false。因此当前图片由主进程同步解码；这个设置最兼容 Windows，也可能让高端 AutoDL GPU 等待 I/O。若要在服务器提高 workers，必须用新的 effective config 记录并先做吞吐/稳定性 A/B，不能把 YAML 中被覆盖的 8 误报成已经生效。

3.8 PairedTransform 为什么必须“成对”

若只翻转 DAPI、不翻转 CD68，标签就错位了。因此水平翻转、垂直翻转、0/90/180/270 度旋转和可选平移会对输入与所有目标同步执行。亮度、对比度、gamma、噪声和模糊则只施加在 DAPI 输入上，用来模拟采集强度变化，同时保留真实 CD68 标签。当前通过 `_make_loader()` 创建普通 PairedTransform 时，只从配置传入 `max_translation`；其余参数使用类的默认值：水平/垂直翻转概率 0.5、旋转概率 1.0、gamma/亮度/对比度概率各 0.5、噪声概率 0.25、模糊概率 0，平移默认为 0。

4 模型架构详解

4.1 先建立整体直觉：这是一个“先压缩理解，再放大还原”的网络

当前初赛 YAML 选择的 MultiMarkerRestorer 可以看成 U-Net 风格的编码器-解码器 [1]。编码器逐级减小宽高、增加通道，把像素转成更抽象的特征；解码器再逐级放大，并从编码器拿回高分辨率 skip feature，恢复细节。与普通 U-Net 不同的是，本模型用 NAF block 负责恢复，在 1/8 瓶颈用共享/任务余弦原型混合语义，并在每个解码尺度用 CD68 task adapter 做条件调制。整体结构如图 7。

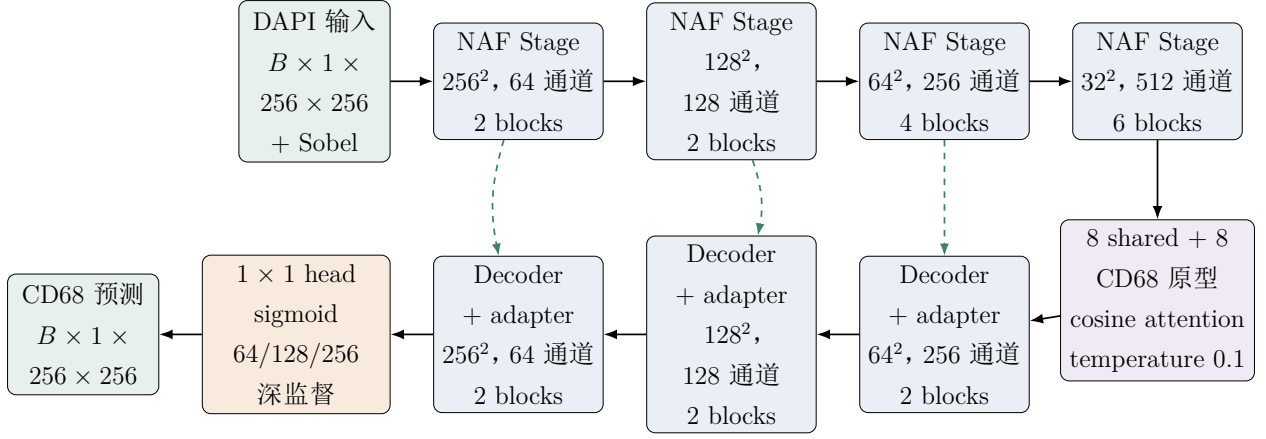


图 7: 当前初赛配置的 MultiMarkerRestorer 主干。图中按实际单逻辑通道标注输入/输出; prototype 与 task adapter 开启, context、cross-attention 和 base/detail 关闭。

4.2 输入预处理与 Sobel 结构提示

当前配置启用 `use_sobel_input=true`。固定 Sobel 卷积核计算水平和垂直梯度，再取

$$S = \sqrt{G_x^2 + G_y^2 + \varepsilon}, \quad (5)$$

得到边缘强度图。当前 Dataset 已把 RGB 存储文件转换为一个逻辑灰度通道，所以 Sobel 直接在这一通道上计算。原输入和 Sobel 图在通道维拼接成 2 通道，再通过保持 256×256 尺寸的 3×3 stem 卷积。Sobel 没有可训练参数，它只是明确告诉模型“哪里变化快”，类似在原图旁边附上一张铅笔勾边图。

4.3 NAF 局部编码器

NAF (Nonlinear Activation Free) 来自 NAFNet 图像恢复思想 [2]。名称的重点是：NAFBlock 主干不依赖 ReLU/GELU 这类常规逐点激活，而使用乘法门控建立非线性。它并不意味着整个模型完全没有激活函数；当前 task adapter 使用 GELU，prototype 分配使用 softmax，最终输出使用 sigmoid。

4.3.1 SimpleGate

若输入特征有 $2C$ 个通道，SimpleGate 沿通道一分为二：

$$\text{SimpleGate}([\mathbf{x}_1, \mathbf{x}_2]) = \mathbf{x}_1 \odot \mathbf{x}_2, \quad \mathbf{x}_1, \mathbf{x}_2 \in \mathbb{R}^{C \times H \times W}. \quad (6)$$

$\odot$ 表示逐元素相乘。直观地说，一半特征像“内容”，另一半像“开关强度”，两者相乘后共同决定信息是否通过。它没有额外参数，但乘法本身产生了非线性关系。

4.3.2 SCA 简化通道注意力

SCA (Simplified Channel Attention) 先对每个通道做全局平均池化, 得到 $C \times 1 \times 1$ 的摘要; 再用 1×1 卷积生成通道缩放系数, 并乘回原特征:

$$\text{SCA}(\mathbf{x}) = \mathbf{x} \odot \text{Conv}_{1 \times 1}(\text{GAP}(\mathbf{x})). \quad (7)$$

它比包含多层全连接和 sigmoid 的传统 SE 模块更简单。这里的“注意力”可理解为模型在问: 当前图像中, 哪些特征通道更值得放大?

4.3.3 NAFBlock 的真实顺序

图 8 按 `naf_blocks.py` 展示两个残差子块。第一支负责空间混合与通道重标定, 第二支像门控前馈层。 β 与 γ 都从 0 初始化, 所以网络刚建立时每个 block 接近恒等映射, 有利于深层模型稳定开始训练。

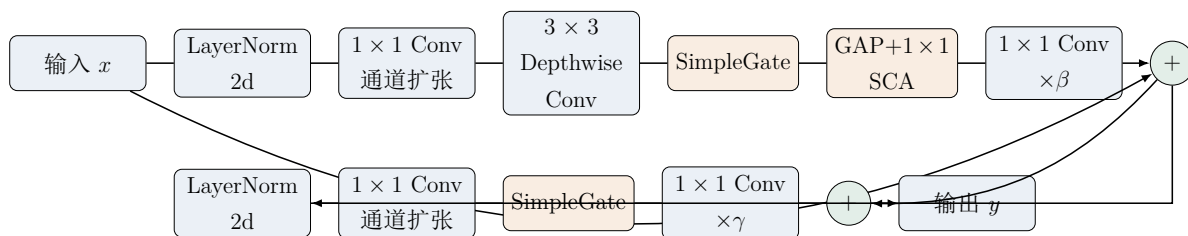


图 8: NAFBlock: 两次归一化、两条门控残差路径和零初始化残差缩放

4.3.4 四个编码尺度

`MultiMarkerRestorer._encode()` 的 stem 保持 256×256 , 每个 NAF stage 之后 (最后一级除外) 再用 2×2 、stride 2 卷积下采样。当前通道为 $[64, 128, 256, 512]$, depths 为 $[2, 2, 4, 6]$:

表 8: 局部编码器各尺度

相对尺度	空间尺寸	通道数	NAFBlock 数	主要作用
1	256×256	64	2	细边缘与局部纹理
1/2	128×128	128	2	中等范围结构
1/4	64×64	256	4	更大感受野与 skip
1/8	32×32	512	6	原型混合前的瓶颈表示

4.4 瓶颈处的共享/任务余弦原型混合

当前主线没有调用 `Restormer-lite`; 它调用的是 `MultiTaskPrototypeMixer`。把 $\mathbf{F} \in \mathbb{R}^{B \times 512 \times 32 \times 32}$ 展平后, 每个空间位置成为一个 512 维 token, 总计 1024 个 token。代码把

token、8 个 shared prototype 和 8 个 CD68 task prototype 都做 L_2 归一化，再计算余弦相似度和温度为 0.1 的 softmax:

$$A_{ik}^{(s)} = \text{softmax}_k \left(\frac{\bar{\mathbf{f}}_i^T \bar{\mathbf{p}}_k^{(s)}}{0.1} \right), \quad A_{ik}^{(t)} = \text{softmax}_k \left(\frac{\bar{\mathbf{f}}_i^T \bar{\mathbf{p}}_k^{(t)}}{0.1} \right). \quad (8)$$

shared 与 task attention 分别对原型加权求和；两个 context 拼接后经线性层投影回 512 维，再以可学习 fusion_scale 残差加回原特征。该尺度初始值为 0.1，所以原型分支从小幅修正开始，而不是一开始覆盖卷积特征。相似度、归一化和 softmax 在 float32 中计算，随后再转回输入 dtype。

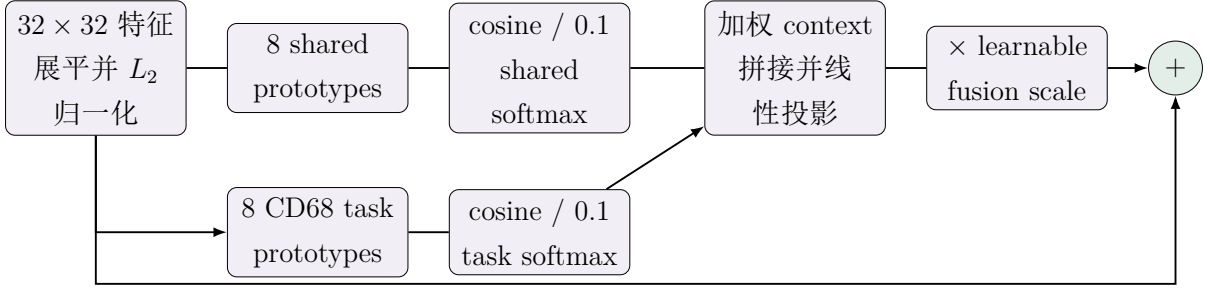


图 9: MultiTaskPrototypeMixer: 共享/任务原型注意力与残差融合

4.5 解码器与 skip connection

解码器有三个 DecoderStage。每一级先用双线性插值把低分辨率特征放大到 skip 的尺寸，再用 1×1 卷积调整通道，把放大特征和编码器 skip 在通道维拼接，经过另一个 1×1 卷积融合，最后用 NAFStage 精炼。skip connection 的作用像给“根据摘要重画原图”的人补充原始草稿：瓶颈保留大局，浅层 skip 带回精细边缘。

三个解码输出尺度分别是 64×64 、 128×128 和 256×256 ；每级 NAFStage 都有 2 个 NAFBlock。每一级随后可经 task adapter，并由独立 1×1 head 产生该尺度的 sigmoid 预测。

4.6 marker conditioning、FiLM 与 adapter

conditioning（条件控制）让共享模型知道当前要预测哪个 marker。当前主模型为目标建立 64 维 task embedding；它不调用 CAMP-VS v2 的 organ conditioner。**FiLM**（Feature-wise Linear Modulation）根据 embedding 生成逐通道缩放和偏置：

$$\text{FiLM}(\mathbf{F}) = (1 + \gamma) \odot \mathbf{F} + \beta. \quad (9)$$

解码器每级的 TaskAdapter 先由 embedding 生成 scale/shift，线性层从零初始化；再经过 LayerNorm2d、depthwise 3×3 、GELU 和 1×1 pointwise 路径，残差尺度也从 0 开始。当前只有 CD68 一个任务，embedding 不能在多个 marker 之间做选择，但接口保留了以后多任务扩展能力。

4.7 直接输出头：当前没有 base/detail 与强度校准

当前 `model.base_detail=false`。所以模型不会调用 `LaplacianBaseDetailHead`，也不会把预测拆成低频 base 和高频 detail；每个解码尺度只是执行 1×1 卷积与 sigmoid：

$$\hat{\mathbf{y}}_s = \sigma(\text{Conv}_{1 \times 1}(\mathbf{F}_s)), \quad s \in \{64, 128, 256\}. \quad (10)$$

当前主线也不实例化 `GlobalIntensityCalibrator`。这两个模块虽然仍存在于可选 CAMP-VS v2 源码中，但不能写成当前较优回退版本的有效组成部分。

4.8 Deep Supervision

深监督（Deep Supervision）是在中间解码层也产生预测，让浅层和深层都直接收到训练信号。当前只有三个输出尺度：256、128、64。损失按照从大到小排序，对它们依次使用 $[1.0, 0.5, 0.25]$ ；目标图用 area 插值缩放到对应大小。全分辨率分支因此获得最大权重。

4.9 当前实际模型规模

工作区没有已经完成训练的 `model_stats.json`，所以本文使用项目自带统计函数，在 CPU 上对当前合并配置做了一次无权重训练的结构测量。结果为 16,232,644 个可训练参数和约 39.160 GMAC/单张 256×256 forward。MAC 是卷积/线性乘加次数估计，不等于精确耗时；实际速度还受到 GPU、数据加载、AMP、显存和验证频率影响。

表 9: 当前单逻辑通道 MultiMarkerRestorer 的顶层参数量

模块	参数量	约占总参数
四个 NAF encoder stages	13,279,616	81.808%
三个 downsample convolutions	689,024	4.245%
shared/task prototype mixer	532,993	3.283%
三层 shared decoder	1,578,752	9.726%
CD68 task adapters	150,528	0.927%
stem、task embedding 与三个 output heads	1,731	0.011%
合计	16,232,644	100%

当前默认启用 shared/task prototypes 与 CD68 task adapters；默认关闭 3×3 ROI context、context cross-attention 和 base/detail。源码中另有 CAMP-VS v2、Restormer-lite、hierarchical prototypes、organ adapter、calibrator 与 MoE 等可选实现，但 `model.name=multi_marker_restorer` 时不会调用它们。报告描述当前主线时不把“文件存在”算作“前向路径已启用”。

5 数据流向详解

5.1 用一个真实 patch 对照源码路径

图 10 继续使用 ROI007_09_11。上方蓝色箭头表示 DAPI 真正进入模型的路径；下方红色分支表示训练时才会读取的 CD68 真值。图中的 CD68 小图是训练标签，不是模型预测。当前报告构建目录没有完成正式训练的 competition checkpoint，因此本文不会拿真值冒充“生成结果”。真正的预测外观必须由指定 checkpoint 在 test DAPI 上运行后产生。

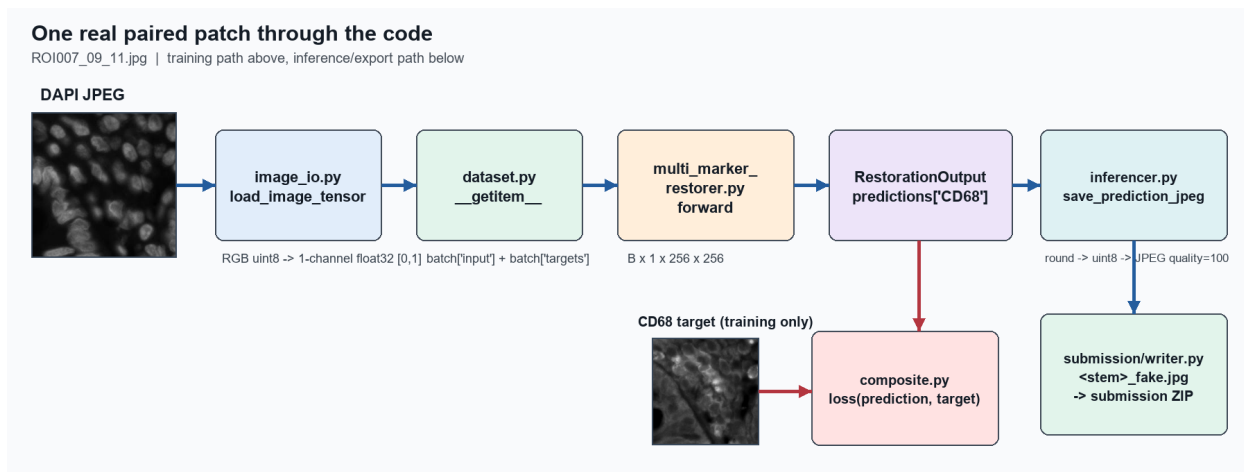


图 10: 一个真实配对 patch 从 JPEG、Dataset、MultiMarkerRestorer 到损失与提交文件的代码路径。

表 10 把图中的每个方框与当前源码对应起来。需要特别注意：四个 target 文件夹不会同时“喂给”单目标 CD68 模型。配置中的 `data.targets` 决定 Dataset 读取哪些真值，`model.name` 决定 CLI 构造哪一个网络；当前初赛配置只读取 CD68 作为监督目标，并构造一个 `MultiMarkerRestorer` 实例。

表 10: 真实 patch 在代码中的表示变化

阶段	输入/数据键	对应源码	产生的结果
图像解码	DAPI/CD68 JPG	utils/image_io.py 的 load_image_tensor()	RGB uint8 HWC 的三个相同通道先取均值并四舍五入, 变为 $[0, 1]$ 的单通道 float32 CHW。
样本组装	manifest 一行	data/dataset.py 的 VirtualStainingDataset. __getitem__()	返回 batch["input"]、batch["targets"]["CD68"] 和 ROI/stem 元数据。
选择模型	合并后的 YAML	cli.py 的 _build_model()	当 model.name=multi_marker_restorer 时实例化 MultiMarkerRestorer, 不会同时运行 CAMP-VS v2 或 U-Net。
前向传播	$B \times 1 \times 256 \times 256$ DAPI	models/multi_marker_restorer.py 的 MultiMarkerRestorer. forward()	NAF 编码、原型混合、task-adapted 解码后返回 RestorationOutput.predictions["CD68"]。
训练比较	预测图与 CD68 真值	losses/composite.py 和 engine/trainer.py	固定权重复合损失、反向传播并更新参数; CD68 真值只走监督分支, 不作为模型输入。
推理保存	test DAPI 与 checkpoint	engine/inferencer.py、 submission/writer.py	预测先保存为 <stem>.jpg, 提交构建时命名为 <stem>_fake.jpg 并写入 ZIP。

从神经网络内部再向下看, `MultiMarkerRestorer.forward()` 不是直接对每个像素套一个公式。它先调用 `_encode()` 得到四尺度 features, 在瓶颈执行 shared/task prototype mixing, 再按任务调用 `_decode_task()`。CD68 分支产生主预测、三尺度深监督和 prototype attention 诊断张量, 最后装入 `RestorationOutput`。训练器取预测与真值算 loss; 推理器只取预测, 执行可选 D4 平均、截断到 $[0, 1]$ 、四舍五入到 uint8, 并以 quality 100、subsampling 0 编码一次 JPEG。提交 writer 随后直接复制已有 JPEG 并改成比赛文件名, 避免第二次有损编码。

5.2 训练数据流

图 11 把训练过程从硬盘一直画到参数更新。张量形状按当前单逻辑通道配置表示。

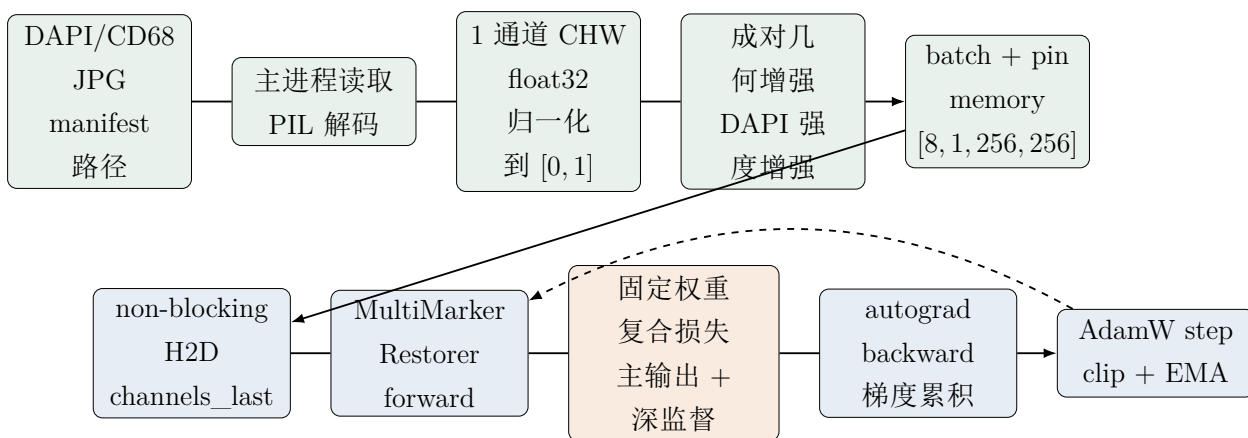


图 11: 训练时从 JPG 到模型参数更新的数据流

进一步拆解如下：

1. 当前有效 `num_workers=0`，所以训练主进程按 manifest 读取配对 JPG，PIL 解码为 RGB uint8；这不是 8 个 worker 并发读取。
2. `load_image_tensor()` 对三个相同 RGB 通道取均值并四舍五入为一个逻辑通道，再转为 CHW float32 并除以 255。
3. `PairedTransform` 同步改变 DAPI/CD68 的几何，仅扰动 DAPI 强度。
4. `DataLoader` 把 8 个样本堆成 batch，页锁定内存为 CPU 到 GPU 传输做准备。
5. `move_to_device()` 对张量使用 non-blocking CUDA copy；训练器在启用时把 4D 浮点输入转为 channels-last 物理布局。
6. `autocast` 选择 BF16 或 FP16 执行适合低精度的算子；模型返回结构化预测结果。
7. `CompositeRestorationLoss` 将预测与 CD68 真值比较，并把 256/128/64 三个分辨率的固定权重损失合并。
8. loss 除以累积因子后反向传播。每两个 `DataLoader` batch 执行一次优化器更新、梯度清零和 EMA 更新。

5.3 验证数据流

验证不更新模型。它要回答的问题是：“用当前权重预测从未用于梯度更新的 ROI，文件质量怎么样？”

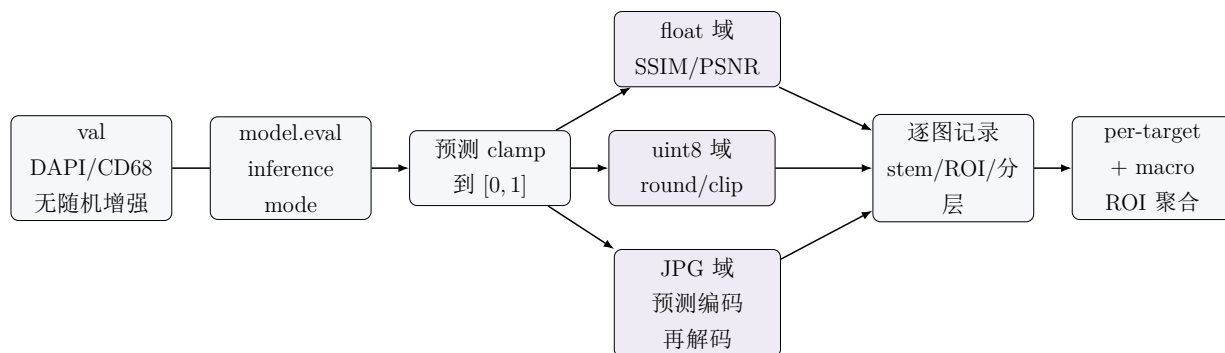


图 12: 验证时的三域指标与 ROI 聚合数据流

验证 loader 不创建 PairedTransform；模型使用 `eval()` 和 `torch.inference_mode()`，所以 dropout 等模块进入推理行为，也不保存梯度图。每张预测先截断到 $[0, 1]$ ，再进入三个指标域。结束后恢复模型原来的 train/eval 状态，避免验证永久改变训练模式。

5.4 测试与推理数据流

测试 manifest 只有 DAPI。Inferencer 保证 loader 不能随机打乱，然后按稳定顺序预测。当前初赛配置为 `tta=none`，所以每张图只做一次 forward；如果以后显式开启 D4 TTA，同一 batch 会经历恒等、 90° 、 180° 、 270° 、水平翻转、垂直翻转、转置和反转置八种变换。每个预测经过精确逆变换后在 float32 中求平均：

$$\hat{\mathbf{y}}_{D4} = \frac{1}{8} \sum_{k=1}^8 T_k^{-1} (f_{\theta}(T_k(\mathbf{x}))) . \quad (11)$$

平均结果 round 到 uint8，并按 checkpoint ImageSpec 指定的 RGB 模式，以 quality 100、subsampling 0、optimize false 编码一次 JPEG，先保存为 `predictions/CD68/<stem>.jpg`。提交构建器再把它直接复制成 `results/test/CD68/<stem>_fake.jpg`，不会把 JPEG 再解码重编码。

推理也有最多三次定向 CUDA OOM 降级：发生明确 CUDA out-of-memory 时把 batch 切成更小 chunk；其他异常不会被误当 OOM 吞掉。

6 训练详解

6.1 epoch 与 batch：把“看很多遍数据”说清楚

epoch（轮次） 表示把当前训练集完整遍历一次。当前有 5004 个训练 patch，batch size 为 8，DataLoader 不丢弃最后不足 8 张的 batch，因此每个 epoch 大约有

$$\left\lceil \frac{5004}{8} \right\rceil = 626 \quad (12)$$

个 batch。当前配置训练 120 epoch，不代表“只看 120 张图”，而是反复遍历数据 120 次，并且每次都得到新的随机增强。

`Trainer.fit()` 是外层 epoch 循环，`train_epoch()` 是内层 batch 循环。每个 epoch 开始时模型进入 train 模式、清空梯度、重置显存峰值统计；结束时记录真实耗时、吞吐量、峰值显存、RSS 内存、学习率、OOM 次数和损失分量。

6.2 一次 batch 内发生什么

对第 t 个 batch，训练器完成：

1. `unpack_batch()` 得到输入、目标字典和元数据；
2. 张量移动到 GPU，并按配置转 channels-last；
3. 在 autocast 中执行模型 forward 与 loss；
4. 检查总 loss 必须是一个有限标量，防止 NaN/Inf 污染参数；
5. 将 loss 除以梯度累积因子并 backward；
6. 到达累积边界后进行梯度裁剪、AdamW step、EMA 更新，并以 set-to-none 方式清空梯度。

6.3 梯度累积与 effective batch

当前 `batch_size=8`、`gradient_accumulation=2`。每个 micro-batch 都做 forward/backward，但连续两个 batch 的梯度先加在一起，再更新一次参数，因此正常情况下

$$B_{\text{effective}} = 8 \times 2 = 16. \quad (13)$$

这并不等价于一次把 16 张图同时放进显存：它用更多计算步骤换取较小显存峰值。每个 epoch 正常约有 $626/2 = 313$ 次 optimizer step，EMA 也大约更新 313 次。

6.4 AMP 混合精度

AMP (Automatic Mixed Precision, 自动混合精度) 让卷积和矩阵乘等适合的算子使用 BF16/FP16，对需要稳定范围的算子保留 float32。这样通常可以减少显存并利用 Tensor Core 加速 [7]。

当前 `amp=true`、`amp_dtype=auto`。CUDA GPU 支持 BF16 时选择 `torch.bfloat16`，否则选择 FP16。GradScaler 只在 CUDA+FP16 时启用；BF16 的指数范围较大，代码不启用 scaler。SSIM、FFT、Sobel gradient 和 prototype cosine 等敏感计算会显式升到 float32，这说明“开 AMP”不是把每个数都粗暴降精度。

6.5 AdamW、学习率、warmup 与梯度裁剪

当前优化器是 AdamW，初始学习率 2×10^{-4} ，weight decay 10^{-4} 。AdamW 将 weight decay 与自适应梯度更新解耦 [5, 6]。前 5 个 epoch 是 warmup，学习率从较小倍数逐步升到基础值；之后使用余弦函数下降：

$$\eta(p) = \eta_0 \frac{1 + \cos(\pi p)}{2}, \quad p \in [0, 1]. \quad (14)$$

warmup 像汽车起步时缓慢踩油门，避免随机初始化阶段更新过猛。每次 optimizer step 前，所有优化参数的梯度总范数被裁剪到 1.0，防止个别 batch 产生异常大的更新。

6.6 EMA：不是另一个独立模型

EMA (Exponential Moving Average, 指数移动平均) 保存一份参数影子：

$$\bar{\theta}_t = 0.999\bar{\theta}_{t-1} + 0.001\theta_t. \quad (15)$$

它不是再训练一遍模型，而是在每个 optimizer step 后平滑跟踪当前参数。验证 EMA 时，代码暂时备份 raw 权重、把 EMA shadow 复制进模型、完成验证后恢复 raw。短训练时 EMA 可能严重滞后，所以当前训练配置明确在每个验证 epoch 同时评价 raw 和 ema，按 JPG SSIM、再按 JPG PSNR 选择更好的来源，而不是先验认定 EMA 一定更优。选中来源会写入 checkpoint 的 extra 元数据，提交时 weight_source=best_jpg 会读取它。

6.7 当前固定权重复合损失

只用像素误差可能得到模糊结果，只用结构指标又可能忽略强度与数值误差。当前配置的 loss.schedule=constant，所以 _build_loss() 构造 CompositeRestorationLoss，不会调用 two-phase 的 ScheduledCompositeLoss。每个输出尺度组合五项重建损失：

$$\mathcal{L}_{\text{Charb}} = \frac{1}{N} \sum_i w_i \sqrt{(\hat{y}_i - y_i)^2 + \varepsilon^2}, \quad (16)$$

$$\mathcal{L}_{\text{SSIM}} = 1 - \text{SSIM}(\hat{\mathbf{y}}, \mathbf{y}), \quad (17)$$

$$\mathcal{L}_{\text{freq}} = \|\log(1 + |\mathcal{F}(\hat{\mathbf{y}})|) - \log(1 + |\mathcal{F}(\mathbf{y})|)\|_1. \quad (18)$$

其中 w_i 是由目标局部均值差异构造的 structure weight；另外还有 MS-SSIM 和 Sobel gradient。Charbonnier 是平滑的 L1 近似，gradient 强调边缘，frequency 比较 FFT 的 log amplitude 而不直接约束容易不稳定的相位。

表 11: 当前固定损失的重建项权重

分量	权重	直观作用
Charbonnier	0.40	带 structure weight 的稳健像素重建
SSIM	0.35	局部结构、亮度与对比度
MS-SSIM	0.10	多尺度结构
Gradient	0.10	Sobel 边缘一致
Frequency	0.05	频谱幅度和纹理能量一致

三个 deep-supervision 尺度各自计算上述五项, 再按 [1.0, 0.5, 0.25] 相加。MSE、pyramid 与 statistics 权重在有效配置中均为 0。跨 marker correlation 的配置权重虽为 0.02, 但当前只有 CD68 一个 prediction, `task_correlation_loss()` 按定义返回图连接的 0, 不会贡献梯度。

prototype activation 与 diversity 两项则**真实启用**, 权重各为 0.001。前者让采样到的瓶颈 token 靠近至少一个 shared/CD68 prototype, 后者惩罚同一 bank 内原型间的平方余弦相似度, 降低所有原型坍缩到同一方向的风险。因此总损失可以概括为

$$\mathcal{L} = \sum_{s \in \{256, 128, 64\}} \lambda_s \mathcal{L}_{\text{recon}, s} + 0.001 \mathcal{L}_{\text{proto-act}} + 0.001 \mathcal{L}_{\text{proto-div}}. \quad (19)$$

6.8 channels-last、cuDNN benchmark 与 float32 matmul precision

当前配置为 CUDA 训练启用 channels-last。逻辑张量形状仍写成 NCHW, 但物理内存更接近 NHWC 排列, 卷积密集模型可能更容易走 Tensor Core 友好的内核。训练模型和输入都会转换; Validator 也转换输入。当前 Inferencer 没有对应的 channels-last 输入转换路径, 因此不能把训练/验证优化自动外推到最终推理。

`deterministic=false`、`cudnn_benchmark=true` 允许 cuDNN 针对固定 256×256 尺寸测试并选择较快卷积算法, 代价是严格逐位复现能力下降。`float32_matmul_precision=high` 在支持的 NVIDIA GPU 上允许 float32 矩阵乘采用更快的内部精度路径 (通常包括 TF32)。这些是性能设置, 不是保证每台 GPU 都获得相同比例加速的魔法开关。

6.9 进度条、异步指标、profiler 与 torch.compile

- `progress_bar=auto`: 仅在交互式终端的 stderr 是 TTY 时显示 tqdm batch 进度, 内容包括 loss、图/秒、学习率和峰值显存; `nohup` 重定向时默认关闭, 避免日志被刷新字符污染。
- `async_metric_logging=false`: 源码实现了把 loss 分量留在 GPU、epoch 末统一同步的异步缓冲, 但当前配置明确关闭, 仍采用每 micro-batch 转 Python float 的兼容路径。不能把“已实现”写成“当前已启用”。

- `profiler.enabled=false`: 可选 profiler 能跟踪最初若干 batch 并导出 Chrome trace, 但当前生产训练不承担这项开销。
- `compile.enabled=false`: 可选 `torch.compile` 失败会回退 eager; 当前配置没有启用, 也不存在“已经通过 compile 加速”的结果。

6.10 OOM 恢复

只有错误明确属于 CUDA out-of-memory 时才进入恢复: 清梯度、清缓存, 把 batch 在内部切成更小 chunk, 同时 microbatch factor 和梯度累积倍增。最多三次; chunk 已经为 1 仍 OOM 时抛出错误。这样能保留近似 effective batch, 又不会把文件损坏、shape 错误等普通 bug 伪装成显存问题。虽然 YAML 有 `max_oom_retries=3`, 当前 Trainer 实现实际把“三次”直接写在判断逻辑中。

6.11 验证频率、checkpoint 与早停

当前有效 `validate_every_n_epochs=1`, 所以每个 epoch 都验证。若以后显式把间隔改大, 最后一个 epoch 仍会强制验证; 跳过验证的 epoch 只更新 `last.ckpt`, 不会冒充 best 或 top-k。

每个 epoch 原子写入 `last.ckpt`。验证指标创新高时分别写 `best_ssim.ckpt`、`best_psnr.ckpt`、`best_proxy.ckpt`。当前 `save_top_k=1`, 所以按 SSIM、PSNR 排序的 top-k 目录只保留一份候选。checkpoint 版本 2 包含模型、EMA、优化器、scheduler、scaler、loss 状态、epoch、global step、完整配置、manifest hash、ImageSpec、目标列表、指标历史、随机数状态和 DataLoader generator 状态, 因此 resume 不是“只加载一份权重重新开始”。

early-stopping patience 为 30, 计数单位是实际验证事件。由于当前每轮都验证, 若连续 30 个验证 epoch 的 SSIM 没有改进, 早停可以在 120 epoch 预算结束前触发。

7 验证详解

7.1 训练和验证的根本差异

训练需要随机增强、梯度和 optimizer; 验证不增强、不建梯度、不开 optimizer。验证数据按 ROI 与训练集隔离, 因此分数衡量的是模型对未见 ROI 的泛化, 而不是对训练样本的记忆。

截至源码核对时, 当前工作区没有可供引用的完整初赛训练 checkpoint、验证 metrics 或 AutoDL 正式日志。因此本报告说明“代码如何计算和选择指标”, 但不填写任何虚构 SSIM/PSNR, 也不声称模型已经达到某个排行榜分数。实际数字必须由附录中的训练与验证命令真实生成。

7.2 SSIM 与 PSNR

代码用 scikit-image 对每张图单独计算。SSIM 的经典形式 [4] 为

$$\text{SSIM}(x, y) = \frac{(2\mu_x\mu_y + C_1)(2\sigma_{xy} + C_2)}{(\mu_x^2 + \mu_y^2 + C_1)(\sigma_x^2 + \sigma_y^2 + C_2)}. \quad (20)$$

μ 比较平均亮度, σ 比较对比度, σ_{xy} 比较结构共同变化。项目数据范围为 1。PSNR 从均方误差得到:

$$\text{MSE} = \frac{1}{N} \sum_i (\hat{y}_i - y_i)^2, \quad \text{PSNR} = 10 \log_{10} \frac{1^2}{\text{MSE}}. \quad (21)$$

若预测与目标完全相同, MSE 为 0, PSNR 合法地为正无穷; 聚合代码会记录无穷值数量。

7.3 float、uint8 与 JPG 三域

float 域 直接比较 $[0, 1]$ 浮点预测与解码后的浮点目标。它反映模型张量本身的理想质量。

uint8 域 预测和目标都按与保存函数相同的 round+clip 规则量化为 $0 \sim 255$, 再除以 255 评分。它衡量 8 位量化损失。

JPG 域 只把预测按 ImageSpec 的 RGB 存储模式、quality 100、subsampling 0、optimize false 编码到内存, 再解码; 目标已经来自官方 JPEG, 只做 uint8 量化, 不再次压缩。它衡量预测最终落盘可能引入的额外误差。

初赛配置设置 `primary_domain=jpg`。这是很合理的本地模型选择代理, 因为提交文件就是 JPEG; 但在没有赛事平台评测代码的情况下, 不能写成“官方评测器一定逐字逐步完全相同”。正确说法是: JPG 域更接近最终提交文件形态, 官方隐藏 test 分数仍只能由上传 ZIP 后的平台给出。

7.4 逐图、ROI、最差样本与分层聚合

每条验证记录包含 target、stem、ROI、organ、坐标、context 可用度、目标图平均值/标准差、activity proxy 和三域 SSIM/PSNR。聚合层输出:

- 图像级 mean/median SSIM、PSNR;
- 每个 ROI 内先平均、再在 ROI 间宏平均的 `roi_ssim/roi_psnr`;
- 最差 10% 图像的均值, 用于发现模型是否只照顾容易样本;
- 按 target 的 per-target 指标与跨 target macro;
- 按 organ、activity、context availability、图像平均亮度、border/interior 的分层结果;

- 本地 proxy: $0.7 \times \text{SSIM} + 0.3 \times \text{截断归一化 PSNR}$ 。

当前 manifest 的 organ 是 `unknown`, 因为目录是 `train/marker` 而不是带器官层级的目录; 所以不要把本地结果描述为 `colon/liver/stomach` 加权评测。坐标解析成功时, Validator 用每个 ROI 实际出现的最小/最大行列标记 `border`, 其余为 `interior`。

7.5 bootstrap 的真实状态

`bootstrap` (自助重采样) 通常指反复有放回抽样, 以估计指标差的置信区间。项目的 `roi_bootstrap_difference()` 会先在每个 ROI 内平均, 再对 ROI 做成对重采样, 默认 1000 次, 输出候选减基线的均值、95% 区间和 `win/tie/loss`。

但是当前 `Validator.evaluate()` 虽然读取了 YAML 中 `bootstrap_by_roi=true` 的意图, 却不会在单次普通验证报告中直接生成 `bootstrap` 区间。`bootstrap` 真正用于 TTA/promotion 比较路径, 例如 `command_validate()` 的 D4 promotion 和性能 v2 消融门。因而“普通 `metrics.json` 已经包含 1000 次 `bootstrap CI`”是不准确的。普通验证包含 ROI 聚合; 候选与基线的置信区间需要走 promotion 比较流程。

7.6 raw、EMA 与 D4 的选择

训练期验证对 raw、EMA 分别完整评分, 然后按 JPG macro SSIM/PSNR 选更优来源。独立 `command_validate()` 加载 checkpoint 时, `best_jpg` 会读取这个来源; 它不是再次自动跑 raw+EMA 两套。若 `inference.tta=d4`, 命令还会运行无 TTA 与 D4, 并利用同一 ROI 清单做 promotion 判断, 把每个 target 的决策写入 `validation/tta_decisions.json`。预测时若该文件存在, 就按 target 应用决策; 不存在时使用配置本身。当前配置值为 `none`, 不会默认执行 D4。

8 测试与提交详解

8.1 为什么 test 不能显示本地分数

test 只有 DAPI, 没有 CD68 真值。SSIM/PSNR 都属于 full-reference metric, 必须同时有预测和真值。因此运行 `autodl-submit` 能得到 1346 张预测和合法 ZIP, 却不能得到 test 分数。平台持有隐藏 CD68 标签, 上传后才返回 official score。把 val 分数叫“test 分数”或用 val 标签冒充 test 都会误导实验判断。

8.2 D4、权重来源与 JPEG 保存

当前推理配置为 `use_ema=false.weight_source=best_jpg.tta=none`。其中 `best_jpg` 优先使用 checkpoint 记录的训练期 raw/EMA 胜者; 如果旧 checkpoint 没有记录, 才根据 `use_ema=false` 回退到 raw, 且 checkpoint 没有 EMA 时也会安全使用 raw。源码支持 D4

的八次 forward，但当前回退版本为了速度默认关闭；只有在 ROI-grouped JPG 验证确实提高后，才应把它改回 D4。

预测 tensor 先 clamp、乘 255、四舍五入为 uint8，按目标 ImageSpec 保存。当前目标审计为 RGB，所以最终模式应为 RGB，而不是 L。

8.3 提交目录与命名

构建后的真实结构是：

```
outputs/initial_round/<run-id>/submission/
|-- results/
|   |-- test/
|       |-- CD68/
|           |-- ROI025_00_00_fake.jpg
|           |-- ...
|           |-- <1346 predictions>
|-- submission_CD68.zip
```

提交命名函数会先剥掉 stem 末尾已有的 `_fake`，防止产生 `_fake_fake.jpg`。ZIP 中每个成员必须从顶层 `results/` 开始，不能多包一层任意文件夹。

8.4 validate-submission 检查什么

validator 从 test manifest 推导 1346 个期望文件名，与实际目录做集合比较，并逐文件检查：

- 缺失、额外和重复输出；
- 扩展名和 `_fake` 后缀；
- 尺寸是否与 manifest 的 256×256 一致；
- JPEG 格式、uint8 dtype、RGB/L mode 和通道数；
- 像素范围、NaN/Inf、全黑和全白；近乎常量图给 warning；
- ZIP 成员是否都在 `results/` 下，是否缺/多文件，CRC 是否通过。

校验成功只代表“提交格式正确”，不代表“模型分数高”。格式校验像考试前检查答题卡是否填全；真正得分还要由平台拿隐藏答案批改。

9 配置系统：怎样读懂并安全修改 YAML

9.1 YAML 是什么

YAML 是一种供人阅读的层级配置格式。它把“经常调整的参数”从 Python 源码中拿出来，避免每次实验都改程序。例如：

```
train:
  epochs: 120
  batch_size: 8
model:
  context:
    enabled: false
```

缩进表示所属关系，不能随意混用 Tab。这里 `train.epochs` 表示训练轮数，`model.context.enabled=false` 表示不启用邻域上下文。布尔量应写成 `true/false`，列表可以写成 `[64, 128, 256, 512]`。

9.2 配置合并优先级

`load_config()` 不是只读一个文件，而是按下列顺序合并；越靠后的值优先级越高：

1. `configs/default.yaml`：所有实验共享的默认值；
2. 用户指定的 YAML，例如 `configs/initial_round_cd68.yaml`；
3. `configs/resolved_local.yaml`：当前文件存在，保存本地审计推导出的数据根、逻辑通道和 worker 设置；
4. CLI 的 `--set key=value`：只覆盖这一次命令，优先级最高。

每个 run 都会把最后生效的配置写入输出目录。复现实验时应保存这个 effective config，而不应只凭记忆记录“我大概用了哪个 YAML”。例如临时跑 3 epoch 可以写：

```
python -m virtual_staining.cli \
  --log-root log \
  autodl-run \
  --config configs/initial_round_cd68.yaml \
  --run-id cd68_quickcheck \
  --max-epochs 3 \
  --skip-submission
```

`--max-epochs 3` 是快速检查，不是正式训练。必须换一个新的 run ID，避免把快速实验和 120 epoch 的正式实验混在一起。

9.3 当前初赛配置逐项说明

表 12 汇总当前真正影响初赛 CD68 训练的主要设置。

表 12: initial_round_cd68.yaml 的关键生效配置

配置项	当前值	含义与影响
data.root	dataset/official	由 resolved-local 配置锁定当前官方数据根。
data.targets	CD68	当前训练只学习 DAPI 到 CD68。
data.num_workers	0	resolved-local 的值优先于初赛 YAML 中的 8；当前由训练主进程同步读取。
logical channels	1 入 / 1 出	RGB 文件内容按审计结论压成一个逻辑灰度通道，提交时再恢复 RGB storage。
model.name	multi_marker_restorer	使用四尺度 NAF、prototype 和 task adapter 主线。
channels/depths	[64,128,256,512] / [2,2,4,6]	控制四级 NAF encoder 容量；stem 保持全分辨率。
global mixer	未启用	当前不运行 Restormer-lite；瓶颈运行 cosine prototype mixer。
context	关闭	当前训练不读取 3×3 邻居，避免未经验证坐标造成泄漏。
prototypes	开启, 8 shared + 8 task	在 32×32 瓶颈执行余弦注意力与残差融合。
task adapters	开启	每个解码尺度使用 CD68 embedding 与零初始化 FiLM/residual adapter。
base/detail / calibrator	关闭 / 未实例化	当前 output head 直接 1×1 卷积后 sigmoid。
loss schedule	constant	使用固定权重 CompositeRestorationLoss，不运行 two-phase schedule。
train.epochs	120	完整训练轮数。
batch / accumulation	8 / 2	正常情况下有效 batch size 为 $8 \times 2 = 16$ 。
optimizer	AdamW	初始学习率 2×10^{-4} ，权重衰减 10^{-4} 。
scheduler	warmup+cosine	前 5 epoch 预热，随后余弦衰减。
AMP	auto	支持时优先 BF16，否则 FP16；FP16 才使用 GradScaler。
EMA	0.999	每个 optimizer step 更新滑动平均权重；验证仍会与 raw 公平比较。

配置项	当前值	含义与影响
validation interval	1 epoch	每个 epoch 跑 float、uint8、JPG 三域验证。
primary domain	JPG	best checkpoint 首先依据更接近提交文件的 JPG 域。
inference TTA	none	当前每图一次 forward；D4 只是可选能力。

9.4 GPU 加速选项与边界

当前配置为 CUDA 训练打开了 `channels_last`、cuDNN benchmark，并把 float32 matmul precision 设为 high。固定 256×256 尺寸适合 cuDNN 自动选择卷积算法。训练器和验证器会把模型与输入转换为 channels-last 内存格式；当前 inferencer 没有同样的显式转换，因此不能笼统声称“训练、验证、推理全流程都使用 NHWC”。`torch.compile`、`profiler` 和异步 metric logging 默认关闭：它们是可实验的性能工具，不应在没有 A/B 计时与数值核实时被描述成已经生效。

9.5 当前通道数的一个重要事实

官方图片的 JPEG storage mode 是 RGB；审计又证明三通道逐像素相同，所以它在信息意义上是灰度图。当前 `resolved_local.yaml` 明确把输入和四个 target 的逻辑通道设为 1。`load_image_tensor()` 读取 RGB 后对三个通道取均值并四舍五入，模型因此构造为 1 通道输入、1 通道输出，参数量与 MAC 统计也以此为准。checkpoint 的 ImageSpec 仍记录原始 RGB storage，保存时把单通道预测复制回三个相同通道；因此“逻辑单通道”与“磁盘 mode L”不是一回事。

9.6 新手修改配置的安全原则

1. 每次只改变一个主要因素，并使用新的 run ID；
2. 先跑 1–3 epoch 检查数据、loss、显存和 checkpoint，再启动 120 epoch；
3. 比较模型时固定 manifest、seed、训练预算和验证协议；
4. 不要根据 test 反复调参，因为 test 没有本地标签；
5. 不要为了“跑起来”把 val 当 test，也不要再 train/val 间混入同一 ROI；
6. 最终选择必须看 JPG round-trip 指标，而不能只看训练 loss。

10 项目文件结构与模块职责

10.1 从根目录看项目

```
project/
|-- configs/                # YAML configurations
|-- dataset/                # data only; never import it into source code
|   |-- official/{train,test}
|-- src/virtual_staining/   # installable Python package
|-- docs/                  # user guides and this report
|-- artifacts/              # manifests and data audit artifacts
|-- outputs/                # checkpoints, validation and predictions
|-- log/                    # compact AutoDL logs and downloadable bundles
|-- pyproject.toml          # package metadata and Python dependencies
`-- README.md               # project entry point for users
```

当前覆盖后的工作区中 `tests/` 目录不存在；上面的树因此没有列出它。源码仍可做 `compileall` 和 `ruff` 静态检查，但不能把“`pytest` 无测试可收集”写成“自动化测试通过”。若此版本要继续长期开发，应从可信快照恢复与当前接口匹配的测试，而不是凭空声称已有回归保护。

源代码采用 `src-layout`（包放在 `src/` 下的 Python 工程布局）。运行 `pip install -e .` 后，Python 能从任意工作目录找到 `virtual_staining`；`-e` 表示 `editable`，修改源码后不必重新复制安装。若完全不安装项目，也可以在项目根目录设置 `PYTHONPATH=src`，但对新手更容易漏设，因此推荐 `editable install`。

10.2 包入口与公共模块

表 13: `src/virtual_staining` 顶层模块

文件	主要职责
<code>__init__.py</code>	定义包版本和公开入口，使目录成为 Python package。
<code>constants.py</code>	集中维护 DAPI、四个目标 marker、别名与文件扩展名等稳定常量。
<code>config.py</code>	读取/深度合并 YAML，解析 CLI 覆盖，严格检查字段和取值，输出 effective config。

文件	主要职责
cli.py	全部命令的总控制器；把数据发现、训练、验证、预测、提交和 AutoDL 日志串成可执行工作流。

10.3 数据模块

表 14: src/virtual_staining/data 模块

文件	主要职责
activity.py	计算/分箱图像活性，用于分层报告或 activity-aware sampler。
audit.py	全量检查尺寸、mode、通道差、值域、配对、相关性和对齐，生成数据审计报告。
dataset.py	根据 manifest 加载 DAPI/目标，转换为 CHW float tensor，支持单/多目标及邻域字段。
discovery.py	自动搜索候选数据根，识别 split/organ/marker 层级并给候选打分。
grouped_folds.py	以真实 ROI/group 为单位构建交叉验证 fold，防止 patch 级随机切分泄漏。
loader_helpers.py	安全构造 DataLoader、worker seed、pin-memory/prefetch 等参数。
manifest.py	配对 DAPI 与各 marker、解析 canonical key/hash/group，生成 train/val/test CSV 和泄漏报告。
neighborhood.py	用行列坐标查找 3×3 邻居，返回 tiles、有效 mask 与相对 offset。
roi_audit.py	检查二维网格方向、空洞、重复坐标、边界连续性以及跨 split 邻域风险。
roi_index.py	严格解析 ROI000_00_00 为 ROI、row、col，并建立坐标索引。
samplers.py	实现均匀或活性相关采样策略。
transforms.py	对 DAPI 和标签同步执行几何增强，对 DAPI 单独执行允许的强度增强。
__init__.py	汇总数据子包的公开符号。

10.4 模型模块

表 15: src/virtual_staining/models 模块

文件	主要职责
baseline_unet.py	轻量 Residual U-Net 基线, 用于 smoke 和回归验证。
camp_vs_v2.py	可选 v2 总模型, 连接 local encoder、global mixer、conditioning、decoder、base/detail 和 calibrator; 当前初赛 YAML 不选择它。
naf_blocks.py	LayerNorm2d、SimpleGate、NAF block 等卷积恢复基础积木。
naf_local_encoder.py	四级局部编码器, 输出 1/2 至 1/16 多尺度特征。
restoration_transformer.py	GPU 友好的 Restormer-lite 通道注意力和门控前馈模块。
task_organ_conditioning.py	marker/organ embedding 与零初始化 FiLM 条件调制。
context_tile_encoder.py	共享参数编码邻居 tile, 结合位置 offset 和有效 mask。
context_fusion.py	将邻域上下文以残差 FiLM 或瓶颈交叉注意力注入中心特征。
hierarchical_prototypes.py	多尺度 shared/marker/organ 原型、用量与 dead-prototype 诊断。
prototype_mixer.py	旧版余弦原型匹配核心, 供兼容模型和扩展模块复用。
laplacian_decoder.py	从低分辨率预测 base、从全分辨率预测 bounded detail, 并执行合成。
intensity_calibrator.py	学习受限的 per-target gain/bias, 校正整体强度而避免无界漂移。
multi_marker_restorer.py	当前有效主模型: 四尺度 NAF 编解码、shared/task prototypes、task adapters 与三尺度输出。
dapi_mae.py	可选 DAPI masked autoencoder 预训练, 不使用标签并需 fold-local 防泄漏。
registry.py	按配置名构造 Residual U-Net、MultiMarker-Restorer 或 CAMP-VS v2, 并统一输出接口。
__init__.py	汇总模型子包接口。

10.5 损失与指标模块

表 16: 损失和指标模块

文件	主要职责
losses/charbonnier.py	平滑且较抗离群值的像素重建损失。
losses/ssim.py	可微 SSIM/MS-SSIM；敏感运算强制 float32。
losses/gradient.py	用 Sobel 梯度约束边缘结构。
losses/frequency.py	在 log-frequency 域比较纹理能量。
losses/pyramid.py	多尺度 Laplacian/pyramid 重建约束。
losses/statistics.py	约束均值、标准差等强度统计。
losses/prototype.py	prototype activation/diversity 等辅助约束。
losses/shift_tolerant.py	可选小位移容忍损失；当前对齐审计为 0，默认关闭。
losses/composite.py	v1 固定权重复合损失和深监督汇总。
losses/scheduled_composite.py	v2 two-phase 权重插值与所有损失项统一入口。
metrics/image_metrics.py	逐图计算 scikit-image SSIM/PSNR。
metrics/domain_metrics.py	float、uint8、JPG 三域量化与 ROI bootstrap 差值。
metrics/aggregation.py	汇总 mean/median、ROI、target、macro、worst-10% 和 proxy。
metrics/promotion.py	按置信区间、分层退化和泄漏状态判断候选能否晋级。
各目录 __init__.py	维护稳定导入接口。

10.6 训练、推理与实验引擎

表 17: src/virtual_staining/engine 模块

文件	主要职责
common.py	公用 DataLoader、模型/损失/优化器构造和配置辅助逻辑。
trainer.py	训练主循环：AMP、累积、裁剪、EMA、OOM 降级、验证、early stopping 和保存。
validator.py	三域逐图推理、分组聚合、raw/EMA 比较和验证产物写入。

文件	主要职责
<code>inferencer.py</code>	无标签确定性预测、D4 逆变换、显存降级和 JPEG 输出。
<code>checkpoint.py</code>	保存/恢复模型、EMA、optimizer、scheduler、scaler、RNG、manifest/config hash。
<code>ema.py</code>	参数指数滑动平均的更新、state dict 与临时应用。
<code>ensemble.py</code>	多 checkpoint 预测组合与通用集成入口。
<code>ensemble_optimizer.py</code>	仅用 OOF/val 预测寻找非负权重，失败时可回退均匀平均。
<code>model_soup.py</code>	对兼容、同血缘模型做权重 soup，并检查安全前提。
<code>multistage.py</code>	管理 multitask、target、organ、metric 等多阶段训练与恢复。
<code>multitask_optimizer.py</code>	equal/FAMO 类多任务权重及梯度冲突处理实验。
<code>pretrainer.py</code>	DAPI-MAE 的 fold-local 预训练流程。
<code>experiment_registry.py</code>	记录实验父子关系、配置、状态和指标，防止选择性汇报。
<code>ablation_budget.py</code>	定义 smoke/screen/confirm/full 预算与 A0-A3 串行约束。
<code>complexity_report.py</code>	统计参数、近似 MAC 和模块占比。
<code>gradient_monitor.py</code>	监控不同任务梯度范数/夹角，诊断负迁移。
<code>prototype_diagnostics.py</code>	导出 prototype 使用率、熵、相似度等诊断。
<code>prototype_monitor.py</code>	训练期跟踪 dead/collapsed prototype，可选生成注意力图。
<code>__init__.py</code>	汇总引擎子包接口。

10.7 提交与工具模块

表 18: 提交和工具模块

文件	主要职责
<code>submission/writer.py</code>	按竞赛目录/命名复制预测，避免二次 JPEG 编码，并创建 ZIP。
<code>submission/validator.py</code>	校验数量、命名、mode、尺寸、像素异常、ZIP 路径和 CRC。
<code>submission/__init__.py</code>	暴露 submission 公共接口。
<code>utils/device.py</code>	发现 CPU/CUDA、显存与 BF16 能力，解析设备和硬件 profile。

文件	主要职责
utils/image_io.py	统一 RGB/L 读取、float/uint8 转换和高质量 JPEG 保存。
utils/logging.py	创建 console/file logger、结构化 JSON 日志和运行元数据。
utils/paths.py	使用 pathlib 处理中文、空格、相对和绝对路径。
utils/seed.py	固定 Python、NumPy、Torch 和 DataLoader 随机种子。
utils/__init__.py	汇总工具子包接口。

10.8 outputs、artifacts 与 log 的区别

- artifacts/ 主要是“数据事实”：发现结果、manifest、审计、泄漏报告和校验和；
- outputs/ 主要是“模型产物”：checkpoint、验证逐图 CSV/JSON、预测和 submission；
- log/ 主要是“方便人和下一位工程师阅读的轻量记录”：终端日志、环境、GPU、吞吐、摘要及可下载 ZIP。

不要把三者混为一谈。删除 checkpoint 可以释放空间，但 manifest 与审计文件很小且决定实验可复现性，通常应该保留。上传给协作者做性能诊断时，优先提供 log bundle、effective config 和 metrics，而不是上传数 GB 的所有 checkpoint；只有需要复现权重行为时才提供选中的 best checkpoint。

11 总结：从一张 DAPI 到竞赛 ZIP

CAMP-VS 把虚拟染色组织成一条可审计而不是“只会跑一个网络”的流水线：先发现数据并把同名 DAPI/CD68 配成 manifest；严格按 ROI 隔离 train/val；把 RGB storage 转换成单逻辑通道 float tensor；用四尺度 NAF 编码器提取结构，在 1/8 瓶颈用 shared/task cosine prototypes 混合，在解码端用 CD68 task adapter 恢复三尺度预测；用 Charbonnier、SSIM、MS-SSIM、gradient、frequency 与 prototype 正则训练；最后在 float、uint8、JPG 三个域验证，选择 raw 或 EMA 权重，以当前无 TTA 配置预测官方 test，并进行严格 ZIP 校验。

项目当前有三个特别重要的边界。第一，本地 val 有 CD68 真值，所以能够计算 SSIM/PSNR；官方 test 只有 DAPI，本地不能算分。第二，文件存储为 RGB，但当前模型实际使用 1 输入/1 输出逻辑通道，提交时才恢复 RGB storage。第三，prototype 与 task adapter 当前确实启用；邻域、Restormer-lite、base/detail、calibrator 和 organ adapter 等模块虽然有源码，但当前初赛配置没有调用。报告不会把“代码存在”写成“默认已使用”。

对初学者而言，最重要的实验习惯不是一次堆叠最多模块，而是保证每次比较公平：固定 split、seed、epoch、三域验证协议和 JPEG 参数，只改一个因素；保存 effective config、日志和逐图指标；任何提升都先在 ROI-grouped val 上重复，再提交平台验证。这样即使一次实验失败，它仍然提供可用证据，而不是变成无法解释的数值。

A AutoDL 从上传到训练的保姆级流程

本附录假设项目已上传到 `/root/autodl-tmp/project`，官方数据位于项目内 `dataset/official/train` 和 `dataset/official/test`，环境名为 `MEDICAL`。如果实际项目路径不同，只改第一条 `cd`；不要修改 Python 源码里的路径。

A.1 第 0 步：确认上传目录

```
cd /root/autodl-tmp/project
pwd
ls
ls dataset/official/train/DAPI | head
ls dataset/official/test/DAPI | head
```

根目录至少应看到 `pyproject.toml`、`src`、`configs`、`dataset`。`train` 下应有 `DAPI` 和 `CD68` 等 marker；`test` 下只有 `DAPI` 是正常的。

A.2 第 1 步：激活环境并让 Python 找到项目

推荐 `editable install`：

```
conda activate MEDICAL
python -m pip install -e . --no-deps
```

`-e` 不会复制一份大型项目，只在环境中登记源码位置；`--no-deps` 表示依赖已经装好，不再下载 PyTorch 等大包。若确实不愿登记项目，每次新终端都必须执行：

```
conda activate MEDICAL
export PYTHONPATH="$PWD/src:$PYTHONPATH"
```

然后验证导入和 GPU：

```
unset OMP_NUM_THREADS
export OMP_NUM_THREADS=$(nproc)
python -c "import virtual_staining; print('package ok')"
python -c "import torch; print(torch.__version__, torch.version.cuda); \
print('cuda=', torch.cuda.is_available()); \
```

```
print(torch.cuda.get_device_name(0) if torch.cuda.is_available() else 'CPU')"
```

若 `cuda=False`，不要启动 120 epoch。先解决 PyTorch/CUDA 环境，否则模型会落到 CPU，训练可能从数小时变成数天。

A.3 第 2 步：可选的 3 epoch 流程检查

第一次上传建议先用独立 run ID 做快速检查：

```
python -m virtual_staining.cli --log-root log autodl-run \
--config configs/initial_round_cd68.yaml \
--data-root AUTO \
--target CD68 \
--run-id cd68_quickcheck \
--max-epochs 3
```

终端会显示 tqdm 进度条和 epoch/batch 进度。这个命令会真实读取、审计、训练、保存和验证，但 3 epoch 结果只证明流程打通。它既不是正式成绩，也不能取代完整训练。

A.4 第 3 步：120 epoch 正式训练与验证

使用一个新的、永不复用的 run ID：

```
python -m virtual_staining.cli --log-root log autodl-run \
--config configs/initial_round_cd68.yaml \
--data-root AUTO \
--target CD68 \
--run-id cd68_full_seed2026
```

一条命令依次执行环境记录、数据发现、manifest、审计、120 epoch 训练和 best checkpoint 验证。不要关闭 SSH 页面后误以为进程一定继续；若需要断线运行，应在 AutoDL 的 tmux 中执行，或使用平台提供的后台任务。

实时观察 GPU 和日志可另开一个终端：

```
watch -n 2 nvidia-smi
tail -f log/cd68_full_seed2026/autodl-run.log
```

具体日志文件名以该 run 的 log/ 目录实际内容为准；`find log -maxdepth 3 -type f` 可列出全部。如果 GPU 利用率偶尔下降但进度条仍前进，可能正在做 JPEG 验证或加载数据，并不等于卡死。长时间为 0% 且 batch 不增长时，再检查日志末尾和系统内存。

A.5 第 4 步：读取验证分数

完整训练结束后，先查看：

```
python -c "import json; \
m=json.load(open('outputs/initial_round/cd68_full_seed2026/validation/metrics.
    json')); \
j=m['domains']['jpg']['macro']; \
print('JPG SSIM =', j['mean_ssim']); \
print('JPG PSNR =', j['mean_psnr']); \
print('ROI SSIM =', j.get('roi_ssim'))"
```

这一步使用 official train 中隔离出的 val:输入和 CD68 真值都存在,因此可以算分。以后改模型时,用相同 manifest、seed、训练预算和这条读取方法比较;新版本 JPG SSIM/PSNR 变高是“本地证据”，还需要逐图和 ROI 分层确认没有集中退化。它并不等于排行榜一定上升。

如需对某个 checkpoint 单独重跑验证，可执行：

```
python -m virtual_staining.cli --log-root log validate \
--config configs/initial_round_cd68.yaml \
--data-root AUTO \
--checkpoint outputs/initial_round/cd68_full_seed2026/checkpoints/best_ssim.
    ckpt
```

A.6 第 5 步：对 test 推理并生成提交包

最稳妥的做法是显式写出刚才选中的 checkpoint：

```
python -m virtual_staining.cli --log-root log autodl-submit \
--config configs/initial_round_cd68.yaml \
--data-root AUTO \
--target CD68 \
--checkpoint outputs/initial_round/cd68_full_seed2026/checkpoints/best_ssim.
    ckpt
```

也可以不写 checkpoint，让程序在 outputs/**/best_ssim.ckpt 中按修改时间自动选最新文件：

```
python -m virtual_staining.cli --log-root log autodl-submit \
--config configs/initial_round_cd68.yaml \
--data-root AUTO \
--target CD68
```

当前 `build_parser()` 没有给 `autodl-submit` 注册 `--run-id` 参数，所以不要照旧版说明在提交命令末尾添加它，否则 `argparse` 会直接报告 “unrecognized arguments”。自动选择在有多个实验时可能拿到错误权重，因此正式提交推荐第一种显式 checkpoint 写法。提交输出目录由 checkpoint 的父 run 目录决定。

`test` 没有 CD68 真值，因此命令只生成预测并检查格式，不会显示真实 SSIM/PSNR。最终上传：

```
outputs/initial_round/cd68_full_seed2026/submission/submission_CD68.zip
```

竞赛平台用隐藏标签评分后，返回的才是 official test score。

A.7 第 6 步：下载哪些文件

表 19: 建议从 AutoDL 下载的产物

文件	用途
<code>log/<run>/...log_bundle.zip</code>	首选诊断包；包含环境、配置、阶段耗时、GPU/资源、指标和日志摘要。
<code>outputs/.../validation/metrics.json</code>	三域汇总指标。
<code>outputs/.../validation/per_image.csv</code>	找最差样本、ROI 和亮度分层退化。
<code>outputs/.../pipeline_report.json</code>	一次流水线的完成状态和关键路径。
<code>outputs/.../checkpoints/best_ssim.ckpt</code>	继续训练或重新推理需要；文件较大，不做诊断时可不传给协作者。
<code>outputs/.../submission/submission_CD68.zip</code>	上传竞赛平台的最终包。

如果要交给另一位工程师阅读并改进，至少提供日志压缩包、汇总/逐图指标、effective config 和平台分数截图/文本。没有这些 provenance，只给一句“跑了 15 小时分数不高”，很难区分瓶颈来自数据加载、模型、loss、验证编码还是环境。

B 常见问题与排错顺序

表 20: 新手常见故障

现象	排查与处理
No module named virtual_staining	确 认 位 于 项 目 根 目 录， 执 行 <code>pip install -e . --no-deps</code> ， 或 正 确 设 置 <code>PYTHONPATH="\$PWD/src:\$PYTHONPATH"</code> 。
CUDA availability 为 False	检查租用实例是否有 GPU、驱动、PyTorch CUDA wheel；修好前不要 CPU 硬跑正式训练。
CUDA out of memory	先关占显存进程；再用 <code>--set train.batch_size=4 --set train.gradient_accumulation=4</code> 保持有效 batch 约 16。训练器也只针对明确 CUDA OOM 最多降级三次。
进度条似乎不动	看 <code>nvidia-smi</code> 、系统内存、日志末尾；数据审计和全量三域验证不是训练 batch，可能暂时没有训练进度条。
DataLoader worker 崩溃	Linux 先减少 <code>data.num_workers</code> ；Windows 本地调试设为 0。检查共享内存、损坏图和 OOM，而非盲目重启。
JPG 指标低于 float	量化和 JPEG 会损失细节，这是三域评估的目的；确认质量 100、subsampling 0 且没有二次编码。
val 很好、平台分数差	检查 ROI 分布差异、过拟合、 <code>submission target/命名、raw/EMA/TTA</code> 选择以及本地指标与官方公式差异。
run ID 已存在	使用新的唯一 run ID；不要覆盖旧 run 后再拿混合文件做比较。
磁盘迅速增大	保留 best checkpoint、日志和指标，清理失败/重复 run 的大 checkpoint 前先核对路径；不要删除官方数据和唯一 best 权重。

C 实现事实核对清单

表 21 把容易混淆的说法与当前代码真实状态并列，便于以后代码升级后重新审阅报告。

表 21: 截至本报告生成时的实现事实

常见说法	当前真实状态
“训练集 6296 张”	6296 是每个 marker 的总配对数；当前 manifest 为 5004 train、1292 val。
“测试集可计算 SSIM”	错。1346 张 test 只有 DAPI，真实标签在平台。

常见说法	当前真实状态
“图像是 L 单通道”	storage 仍为 RGB; resolved-local 让模型实际使用 1 入/1 出逻辑通道, 提交时恢复 RGB。
“默认用了 3×3 邻域”	错。context 实现存在, 初赛配置为 disabled。
“默认用了 prototype”	对。当前启用 8 个 shared 与 8 个 CD68 task prototypes, 并有两项 0.001 正则。
“NAF 模型完全没有任何激活”	NAF block 的核心是 activation-free; 全模型其他分支仍可包含 GELU 等操作。
“EMA 一定比 raw 好”	错。每个验证点同时评分, 由 JPG 指标选择; 短训练 EMA 可能滞后。
“验证只按 YAML domains 列表算一域”	当前 Validator 实际总是生成 float、uint8、JPG 三域。
“JPG 域等于官方评分”	它最接近提交文件, 但官方公式/隐藏处理不在本地代码中, 不能宣称完全一致。
“普通 metrics.json 含 bootstrap CI”	普通验证有 ROI 聚合; 成对 CI 在 promotion/TTA 比较路径生成。
“早停 30 表示固定跑满 120 epoch”	错。当前每轮都验证; 连续 30 个验证事件无 SSIM 改进即可提前停止。
“所有阶段都用了 channels-last”	Trainer/Validator 显式使用; 当前 Inferencer 没有同样的显式转换。
“当前模型是 37.35M Transformer”	错。实际为 16,232,644 参数的 MultiMarker-Restorer; 无 Restormer forward, 近似 39.160 GMAC。

参考文献

- [1] O. Ronneberger, P. Fischer, and T. Brox, “U-Net: Convolutional Networks for Biomedical Image Segmentation,” *Medical Image Computing and Computer-Assisted Intervention*, 2015. <https://arxiv.org/abs/1505.04597>
- [2] L. Chen, X. Chu, X. Zhang, and J. Sun, “Simple Baselines for Image Restoration,” *European Conference on Computer Vision*, 2022. <https://arxiv.org/abs/2204.04676>
- [3] S. W. Zamir et al., “Restormer: Efficient Transformer for High-Resolution Image Restoration,” *IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2022. https://openaccess.thecvf.com/content/CVPR2022/html/Zamir_Restormer_Efficient_Transformer_for_High-Resolution_Image_Restoration_CVPR_2022_paper.html

- [4] Z. Wang, A. C. Bovik, H. R. Sheikh, and E. P. Simoncelli, “Image Quality Assessment: From Error Visibility to Structural Similarity,” *IEEE Transactions on Image Processing*, vol. 13, no. 4, pp. 600–612, 2004. <https://doi.org/10.1109/TIP.2003.819861>
- [5] I. Loshchilov and F. Hutter, “Decoupled Weight Decay Regularization,” *International Conference on Learning Representations*, 2019. <https://arxiv.org/abs/1711.05101>
- [6] PyTorch Contributors, “AdamW—PyTorch Documentation.” <https://docs.pytorch.org/docs/stable/generated/torch.optim.AdamW>
- [7] PyTorch Contributors, “Automatic Mixed Precision Package—PyTorch Documentation.” <https://docs.pytorch.org/docs/stable/accelerator/amp.html>
- [8] Thermo Fisher Scientific, “DAPI Stain.” <https://www.thermofisher.com/us/en/home/life-science/cell-analysis/fluorophores/dapi-stain.html>
- [9] The Human Protein Atlas, “CD68 tissue expression and primary data.” <https://www.proteinatlas.org/ENSG00000129226-CD68/tissue/primary+data>
- [10] R. Gottfried et al., “Expression of CD68 in non-myeloid cell types,” *Scandinavian Journal of Immunology*, vol. 67, no. 5, pp. 453–463, 2008. <https://pubmed.ncbi.nlm.nih.gov/18405323/>
- [11] D. L. Farber, N. A. Yudanin, and N. P. Restifo, “Human memory T cells: generation, compartmentalization and homeostasis,” *Nature Reviews Immunology*, vol. 14, pp. 24–35, 2014. <https://pmc.ncbi.nlm.nih.gov/articles/PMC4032067/>
- [12] L. J. Stern and J. M. Calvo-Calle, “HLA-DR: molecular insights and vaccine design,” *Current Pharmaceutical Design*, vol. 15, no. 28, pp. 3249–3261, 2009. <https://pmc.ncbi.nlm.nih.gov/articles/PMC3615543/>
- [13] The Human Protein Atlas, “VIM protein expression summary.” <https://www.proteinatlas.org/ENSG00000026025-VIM>
- [14] J. C. Waters, “Accuracy and precision in quantitative fluorescence microscopy,” *Journal of Cell Biology*, vol. 185, no. 7, pp. 1135–1148, 2009. <https://pmc.ncbi.nlm.nih.gov/articles/PMC2712964/>